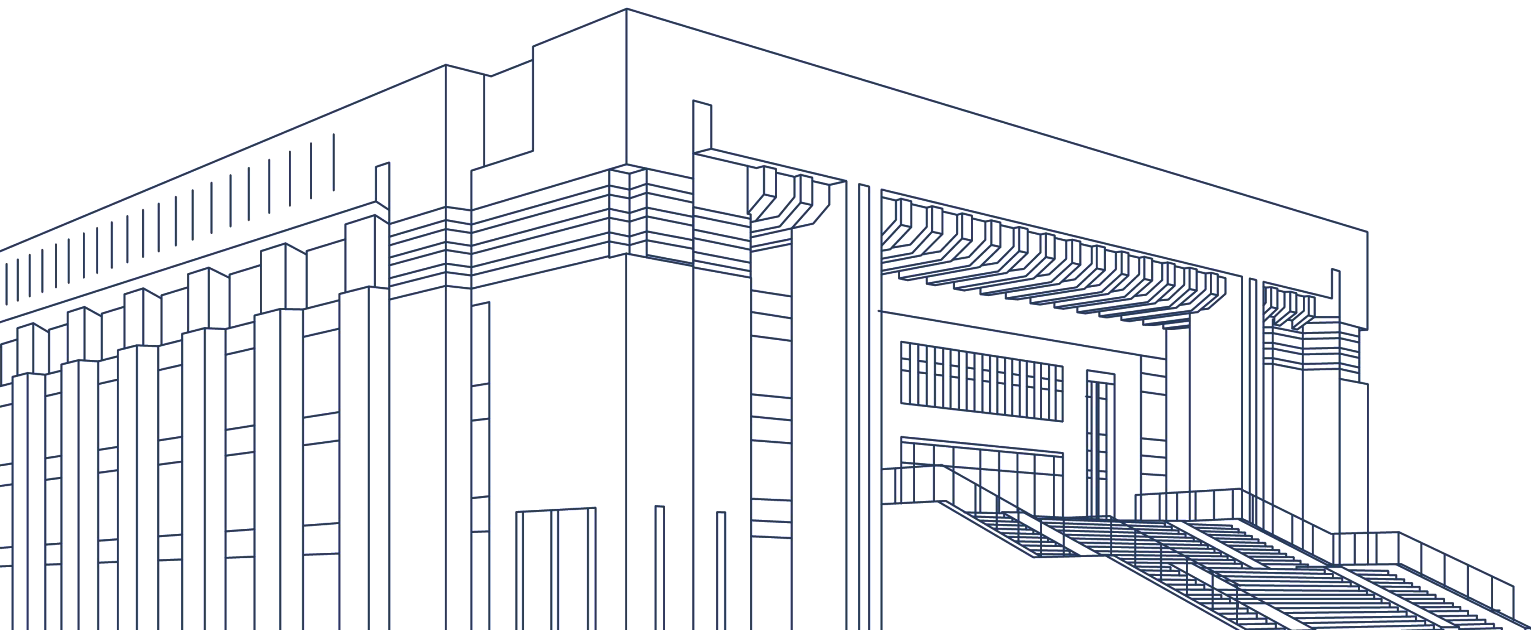




河北工程大学
Hebei University of Engineering

大学计算机（1）

计算思维与信息技术





第4章 算法思维——问题求解的核心



4.1 算法的概念



4.2 算法的设计与分析



4.3 算法的实现——程序设计语言



主要内容

- 理解计算机问题求解中的数学建模。
- 算法的概念。
- 算法的特征。
- 理解常用的算法设计策略以及算法的描述方法。





4.1 算法的概念



4.1.1 什么是算法

为解决一个问题而采取的方法或步骤，同一个问题可以有不同的算法，不同的算法效率不同。

4.1.2 算法的分类

- (1) 生活算法
- (2) 数学算法
- (3) 计算机算法



4.1 算法的概念

(1) 生活中的算法

【例1】农夫过河问题

在河的一岸，一个农夫带着一只狼、一只羊和一颗卷心菜，农夫要将它们运到河的对岸，但由于他的船太小，每次只能载一样东西。并且，当农夫不在时，**狼会把羊吃掉，而羊又会把卷心菜吃掉**。问农夫如何将它们安全渡过河去？



步骤如下：

- ① 把羊运过河
- ② 把菜运过河
- ③ 把羊捎回
- ④ 把狼运过河
- ⑤ 把羊运过河



4.1 算法的概念

(2) 数学算法

对于一类计算问题的机械的、统一的求解方法。

(3) 计算机算法

对运用计算思维设计的问题求解方案的精确描述。计算机算法能够帮助我们解决很多问题。推荐算法、路径规划算法、加密算法、排序算法、遗传算法、贪心算法等。



4.1 算法的概念

4.1.2 算法的特征

1. 确切性

算法的每个步骤必须有确切的定义，无二义性。

2. 可行性（有效性）

算法中任何步骤都可分解为基本的可执行的操作步骤。

3. 输入项

一个算法有零个或零个以上的输入。

4. 输出项

一个算法必须有一个或一个以上的输出。

5. 有穷性

一个算法必须在执行有限步后终止。



4.2 算法的设计与分析

4.2.1 问题求解的步骤

(1) 建立现实问题的数学模型。

通过对问题的抽象，建立**数学模型**（运用数学表达式描述内在规律）或概念模型、功能模型。

(2) 确定输入/输出。

(3) 算法设计与分析。

算法设计是设计一组将数学模型中的数据进行操作和转换的步骤；
算法分析是计算算法时间复杂度和空间复杂度。



4.2 算法的设计与分析

4.2.2 数学建模

数学建模是运用**数学的语言和方法**，通过抽象、简化，建立对问题进行精确描述和定义的数学模型，即问题**抽象**，并用数学语言进行形式化描述。

一些表面上看是非数值的问题，经过进行数字形式化后，可方便地进行算法设计。



4.2 算法的设计与分析

4.2.2 数学建模

4-1:

国际会议排座位问题

现要举行一个国际会议，有7个人分别用a、b、c、d、e、f、g表示。已知各人掌握语言情况如下，这7个人应如何排座位，才能使每个人都能和他身边的人顺利地沟通交谈？

人员	a	b	c	d	e	f	g
语种	英语	汉语 英语	俄语 意大利语 英语	汉语 日语	意大利语 德语	俄语 日语 法语	德语 法语

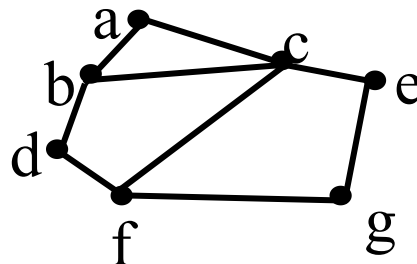


4.2 算法的设计与分析

4.2.2 数学建模

	a	b	c	d	e	f	g
语种	英语	汉语 英语	俄语 意大利语 英语	汉语 日语	意大利语 德语	俄语 日语 法语	德语 法语

尝试建立一个图的模型，将每个人抽象为一个结点，得到结点集合 $V = \{a, b, c, d, e, f, g\}$ 。对于任意的两点，若有共同语言，就在它们之间连一条无向边，得到边的集合 $E = \{ab, ac, bc, bd, df, cf, ce, eg, fg\}$ ，图 $G = \{V, E\}$ 。



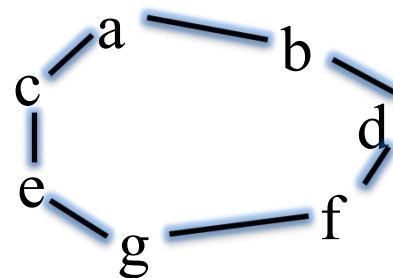
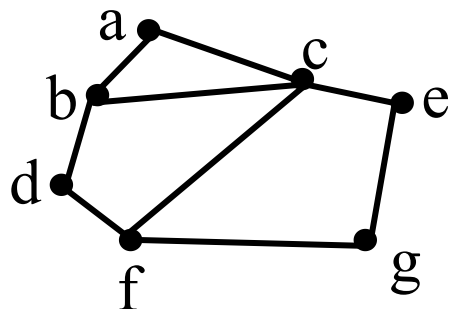


4.2 算法的设计与分析

4.2.2 数学建模

此问题转化为在图G中找到一条哈密顿回路的问题。

- 哈密顿图是一个无向图，是指从图中的任意一点出发前往指定的终点，经过图中每一个结点当且仅当一次——一笔画问题。
- 闭合的哈密顿路径称作“哈密顿回路”。
- “**a**abdfge**a**”是一条哈密顿回路，照此顺序排座位即可满足问题要求。



哈密顿回路



4.2 算法的设计与分析

4.2.2 数学建模

4-2

警察抓小偷

警察局抓了a, b, c, d四名偷窃嫌疑犯，其中只有一人是小偷。四个人中三人说的是真话，一人说的是假话，审问记录如下，谁是小偷？



a说：“我不是小偷。”

b说：“c是小偷。”

c说：“小偷肯定是d。”

d说：“c在冤枉人。”



4.2 算法的设计与分析

4.2.2 数学建模

问题分析：依次假设每个人是小偷，代入四句话中，并检验“**四个人中三人说的是真话，一人说的是假话**”是否成立，如果成立，那么对应的假设成立，即小偷找到。

❖ 算法设计思路：

- ① a,b,c,d四人用'a', 'b', 'c', 'd'表示，变量x存放小偷的编号，x的初值为'a'；
- ② 依次将x='a',x='b',x='c',x='d'代入，检验“四个人中三人说的是真话，一人说的是假话”是否成立。



4.2 算法的设计与分析

❖ 算法设计:

a说: “我不是小偷。”

b说: “c是小偷。”

c说: “小偷肯定是d。”

d说: “c在冤枉人。”

• 四个人中三人说的是真话，一人说的是假话

$x \neq 'a'$

$x = 'c'$

$x = 'd'$

$x \neq 'd'$

关系表达式的值:

- ◆ 成立 (1)
- ◆ 不成立 (0)

■ 四个逻辑式的值相加
 $= 1 + 1 + 1 + 0$
 $= 3$

即: $(x \neq 'a') + (x = 'c') + (x = 'd') + (x \neq 'd') = 3$



4.2 算法的设计与分析

算法如下：

- (1) 初始化： $x = 'a'$;
- (2) x 从 'a' 循环到 'd'；
- (3) 对于每一个 x ，依次进行检验：如果 $(x \neq 'a') + (x = 'c') + (x = 'd') + (x \neq 'd')$ 的和为 3，则输出结果并退出循环，否则继续下一次循环。

一些表面上看是非数值的问题，
经过数字化后，就可方便地进行算法设计。



4.2 算法的设计与分析

使用Python代码实现：

```
for x in ['a','b','c','d']:  
    if (x!='a')+(x=='c')+(x=='d')+(x!='d')==3:  
        print('小偷是:', x)
```

```
===== RES1  
小偷是: c  
>>>
```



4.2 算法的设计与分析

4.2.3 算法的描述

描述方式：自然语言、流程图、盒图、伪代码、程序设计语言。

1. 自然语言

例4-3 输入圆半径，计算圆的面积。

第1步：输入圆半径 r ；

第2步：计算 $s=3.14 \times r \times r$ ；

第3步：输出 s 。

优点：通俗易懂。

缺点：歧义性可能导致不确定性；算法复杂时很难清晰地表示出来；不便翻译成计算机语言。



4.2 算法的设计与分析

2. 流程图

流程图最早出现在上世纪20年代，主要应用在工业领域、制造业中，用图示的方法来表示过程。1950年开始，流程图被广泛地应用于软件设计中。

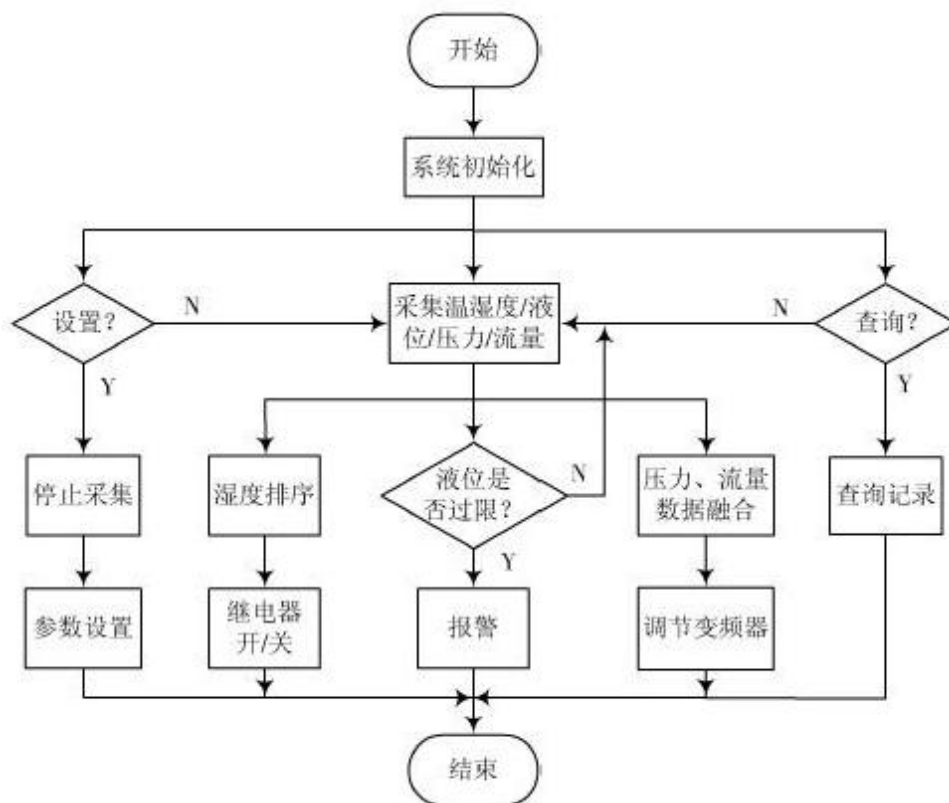


图4.4 程序流程图



4.2 算法的设计与分析

4.2.3 算法的描述

2. 流程图

程序流程图是统一规定的标准符号描述程序运行具体步骤的图形表示。

流程图的设计是在处理流程图的基础上，通过对输入输出数据和处理过程的详细分析，将计算机的主要运行步骤和内容标识出来，是进行程序设计的最基本依据。

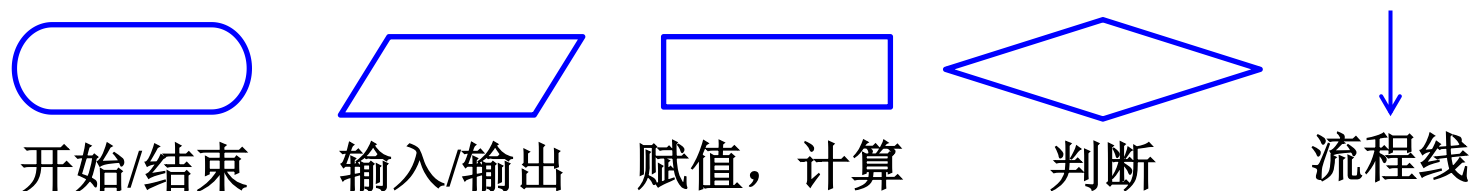


图4.5 程序流程图常用图形符号



4.2 算法的设计与分析

■ 程序设计语言的基本控制结构

1996年，计算机科学家Boehm和Jacopini提出并从数学上证明，任何一个算法，都能以三种基本控制结构表示：顺序结构、选择结构和循环结构。

◆ 顺序结构

最基本和最简单的结构，按书写顺序，依次执行语句1、2...、语句n。

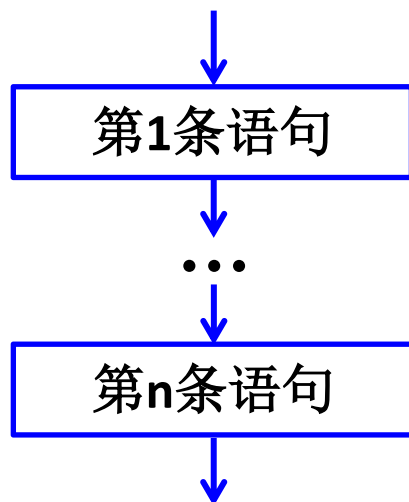


图4.6 顺序结构的流程图



4.2 算法的设计与分析

■ 程序设计语言的基本控制结构

◆ 选择结构

又称分支结构（单分支、双分支和多分支）。它是根据判定条件的真假的来确定应该执行哪一条分支的语句序列。

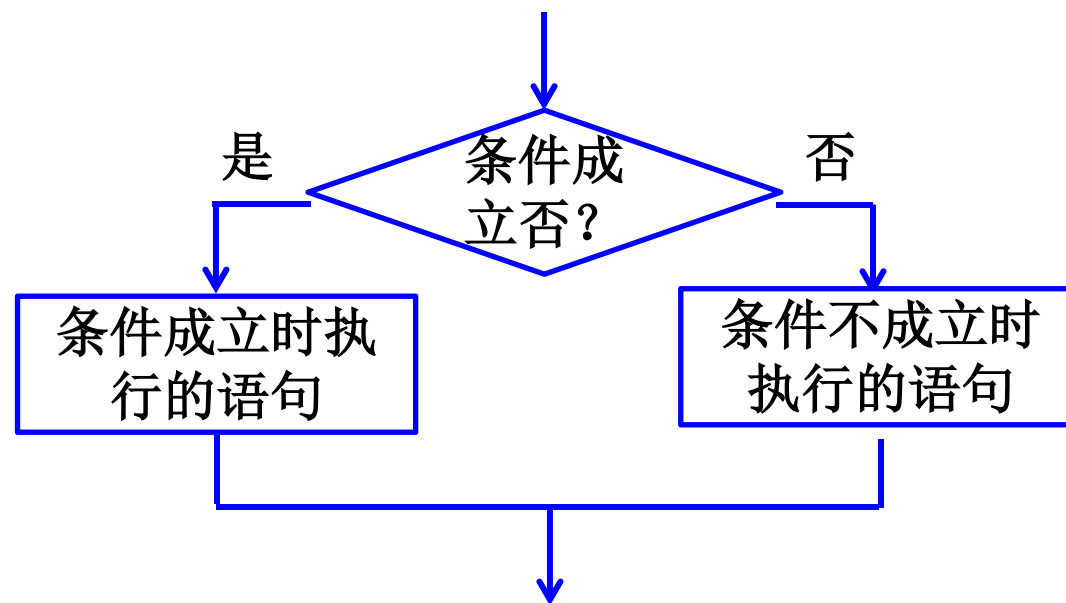


图4.7 分支结构的流程图



4.2 算法的设计与分析

■ 程序设计语言的基本控制结构

◆ 循环结构

使计算机在一定条件下**反复多次**执行同一段程序（称为循环体），从而简化程序。

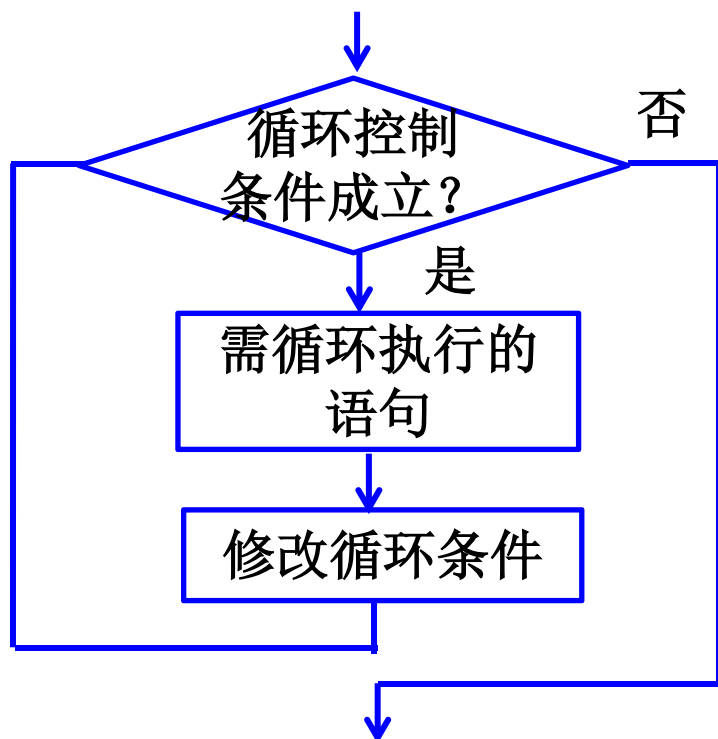


图4.8 有界循环结构流程图

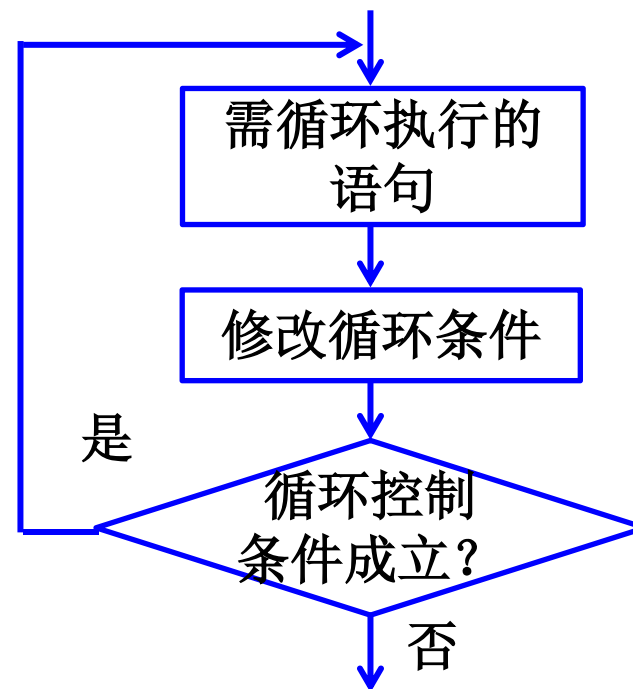


图4.9 条件循环结构流程图



4.2 算法的设计与分析

4.2.3 算法的描述

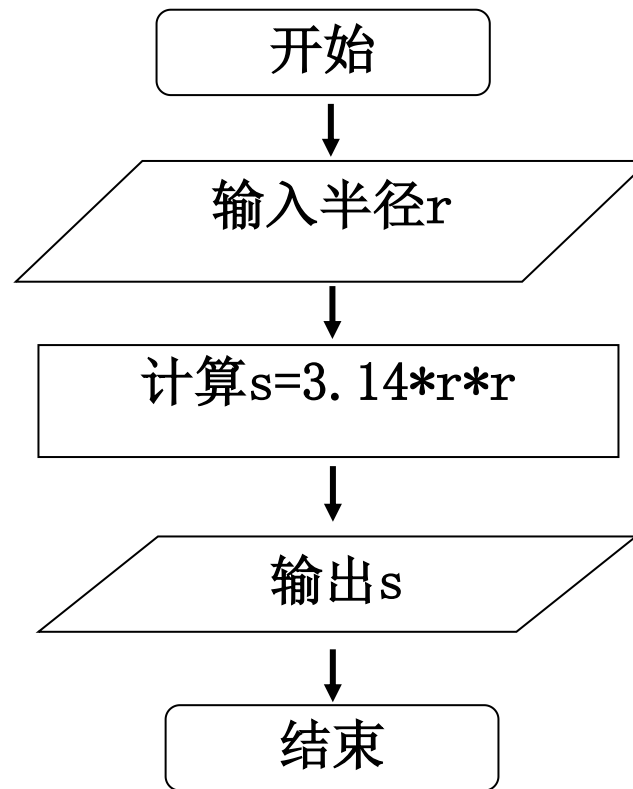
例4-4 输入圆半径，计算圆面积。使用流程图描述。

自然语言描述为：

第1步：输入圆半径 r ；

第2步：计算 $s=3.14 \times r \times r$ ；

第3步：输出 s 。



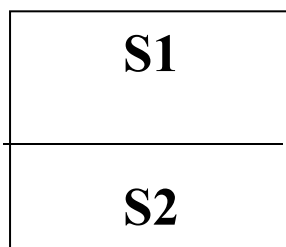


4.2 算法的设计与分析

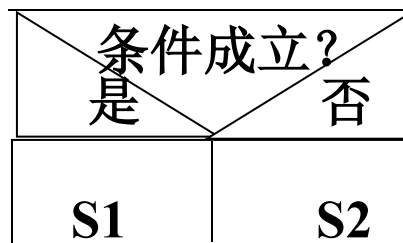
4.2.3 算法的描述

3. 盒图（N-S图）

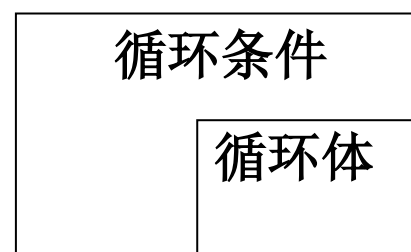
去掉流程线，更加简洁。



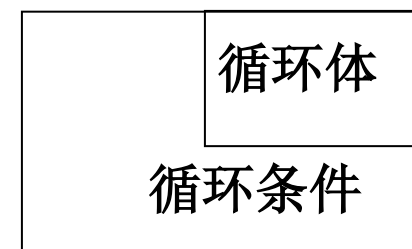
顺序结构



分支结构



循环结构1



循环结构2



4.2 算法的设计与分析

4.2.3 算法的描述

4.伪代码

- 非正式语言：
 - 移植编程语言书写形式作为基础和框架。
 - 按照接近自然语言的形式表达算法过程。
- 兼顾自然语言与编程语言优势：
 - 简洁表达算法本质，不拘泥于实现细节。
 - 准确反映算法过程,不产生矛盾和歧义。

```
输入: 数组  $A[a_1, a_2, \dots, a_n]$ 
输出: 升序数组  $A'[a'_1, a'_2, \dots, a'_n]$ , 满足  $a'_1 \leq a'_2 \leq \dots \leq a'_n$ 
for  $i \leftarrow 1$  to  $n - 1$  do
     $cur\_min \leftarrow A[i]$ 
     $cur\_min\_pos \leftarrow i$ 
    for  $j \leftarrow i + 1$  to  $n$  do
        if  $A[j] < cur\_min$  then
            //记录当前最小值及其位置
             $cur\_min \leftarrow A[j]$ 
             $cur\_min\_pos \leftarrow j$ 
        end
    end
    交换  $A[i]$  和  $A[cur\_min\_pos]$ 
end
```

选择排序算法描述



4.2 算法的设计与分析

【算法设计练习1】输入3个不等的整数，输出最大值，试画出流程图。

step1: 输入3个数 x, y, z ;

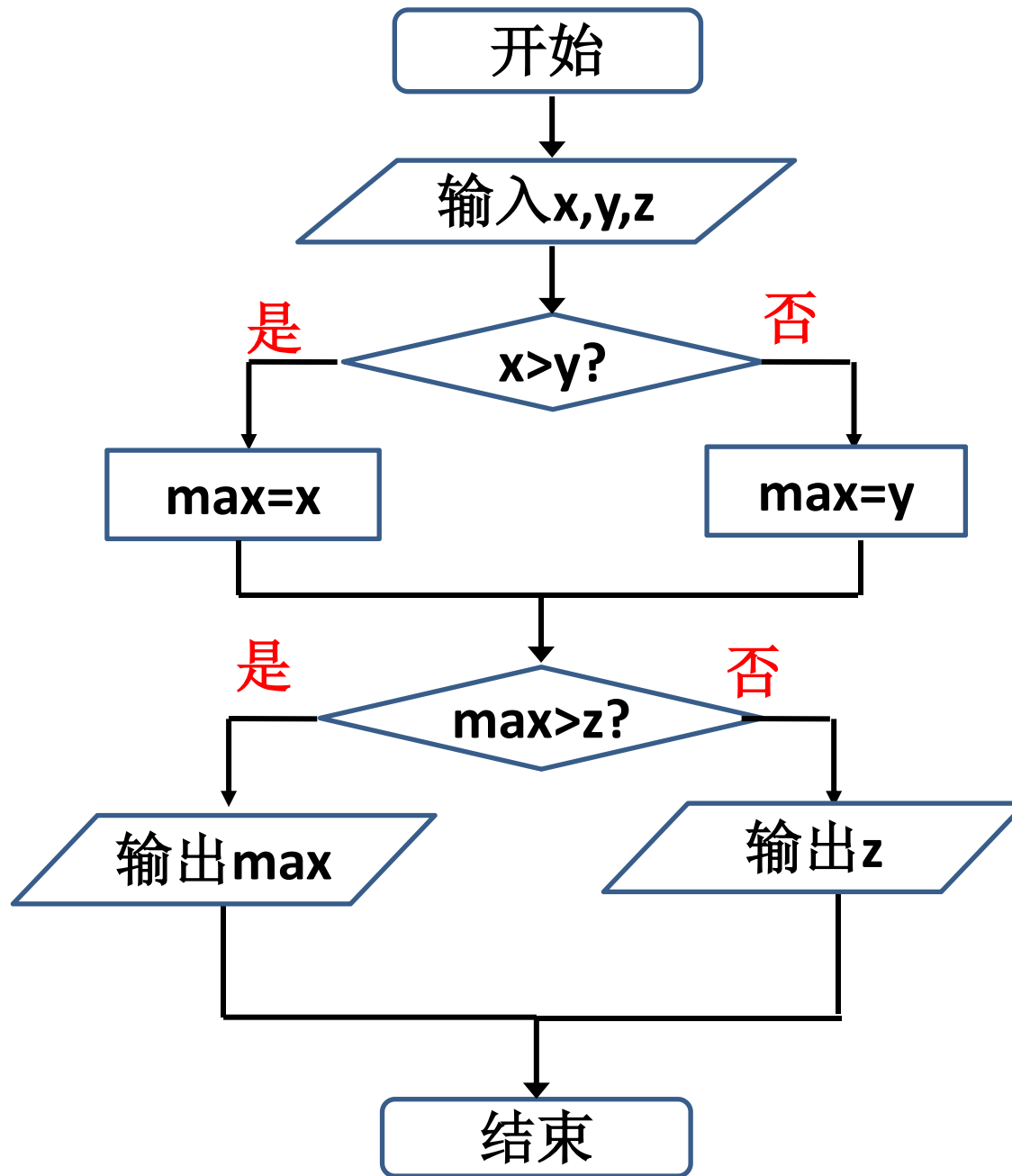
step2: 比较 x 和 y 大小，如果 $x > y$ ，将 x 存至 $\max$ 中，否则将 y 存至 $\max$ 中;

step3: 比较 $\max$ 和 z 的值，如果 $\max$ 比 z 大，输出 $\max$ ，否则输出 z 。

step4: 算法结束。

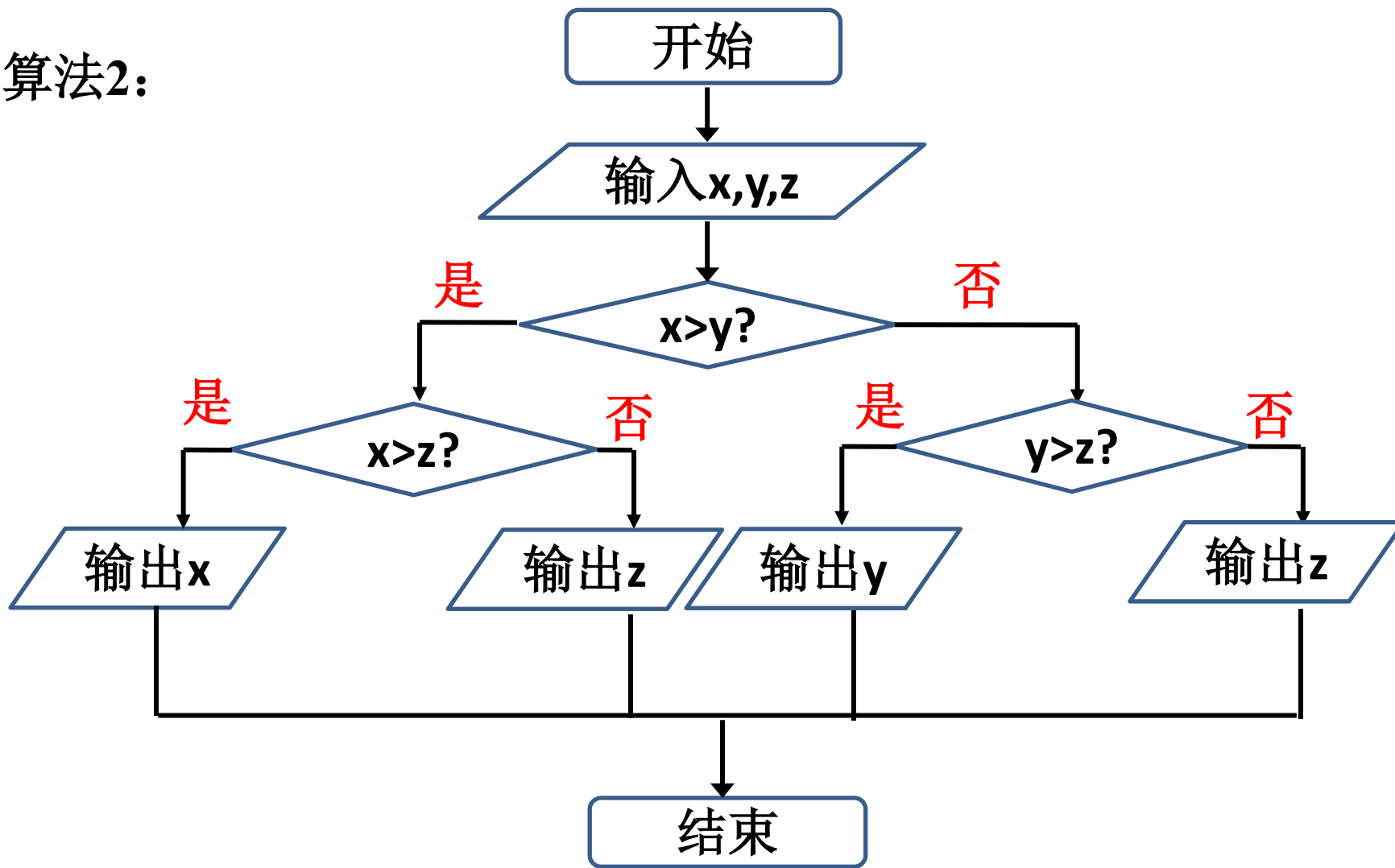


算法1:





算法2:





4.2 算法的设计与分析

【算法设计练习2】 计算 $1+2+3+\dots+100$ ，试画出流程图。

分析：

设变量s初值为0，变量i存放第一个加数1，s和i相加后将和重新存至s中，变量i的值增1，再与s求和并存至s中，依此类推，直到i的值大于100。

step1: $s=0$

step2 : $i=1$

step3 : 如果i小于等于100，执行下一步；否则执行Step6;

step4 : $s+i \rightarrow s$

step5 : $i+1 \rightarrow i$ ，返回step3;

step6: 输出s的值;

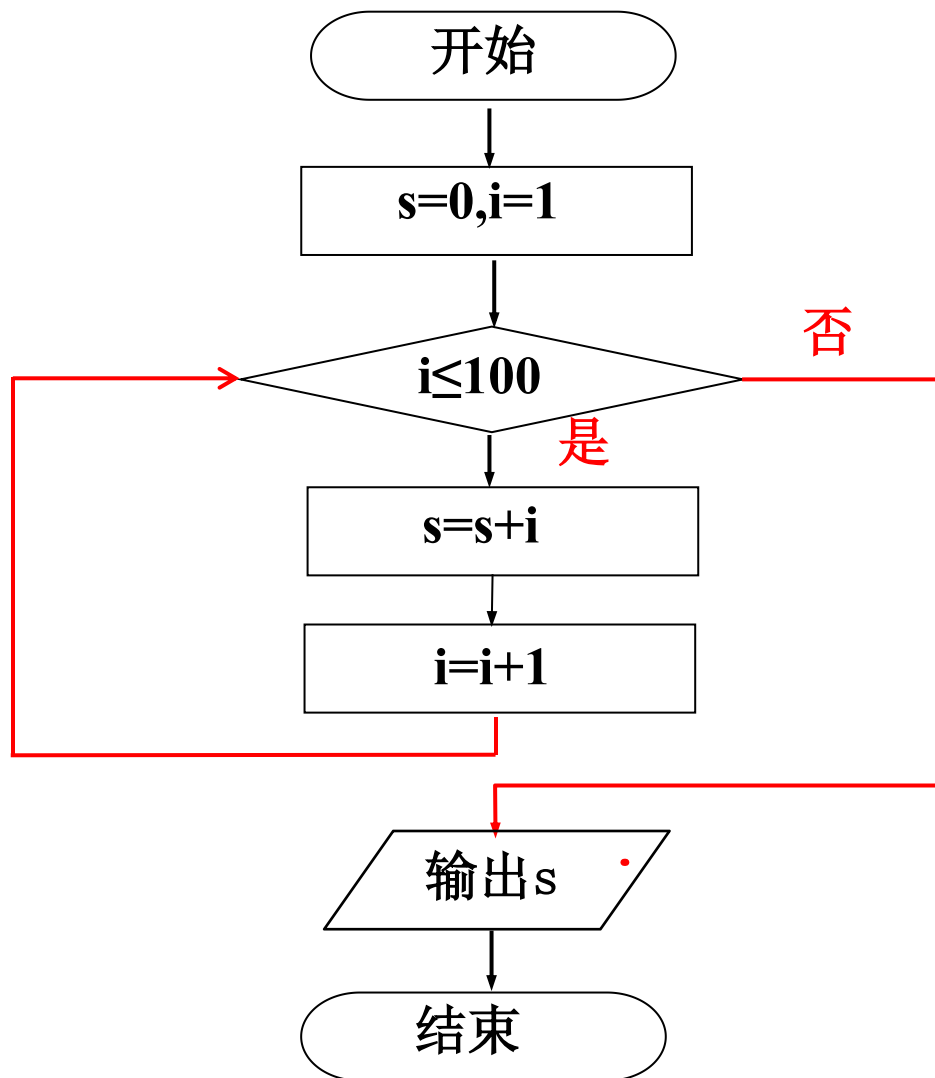
step7: 算法结束。



4.2 算法的设计与分析

【算法设计练习2】计算 $1+2+3+\dots+100$ ，试画出流程图。

流程图：





4.2 算法的设计与分析

【算法设计练习3】计算5! 试画出流程图。

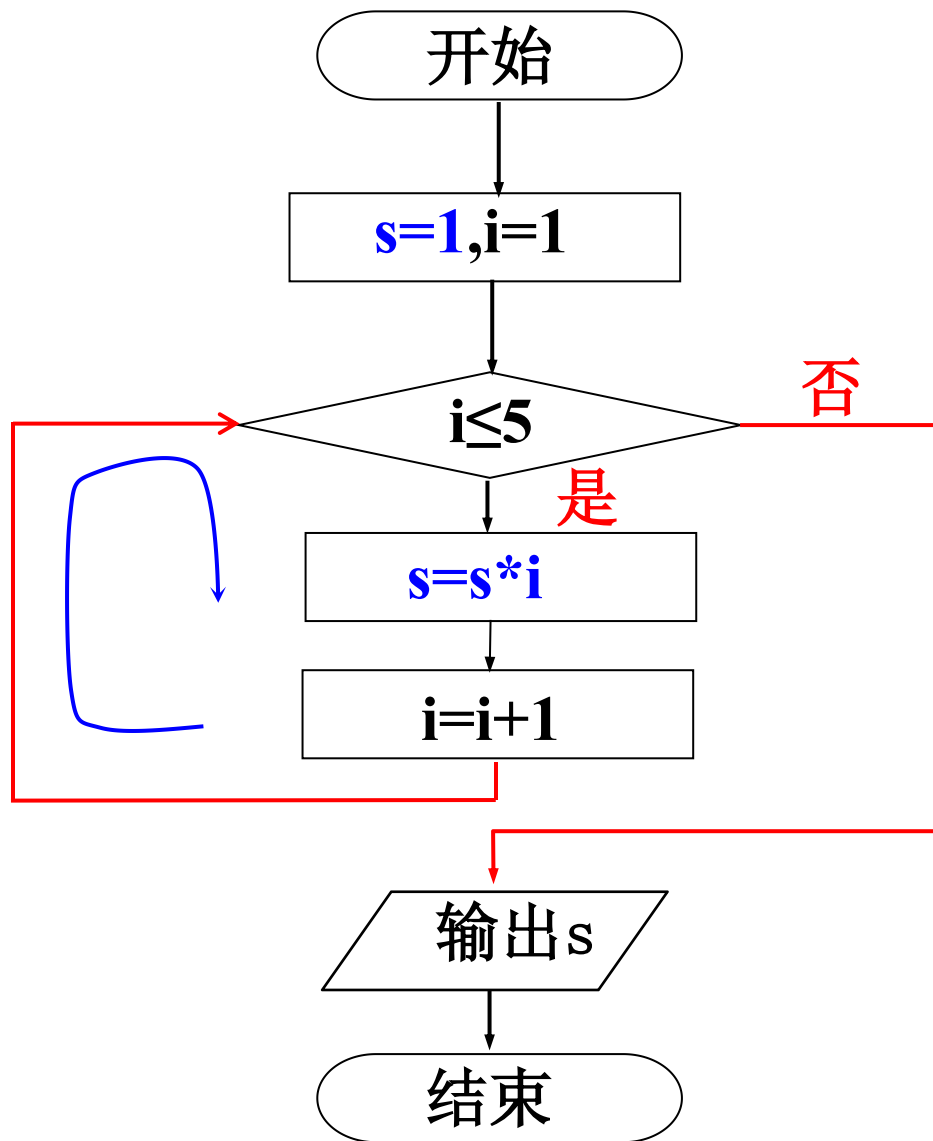
分析：设变量s初值为1，变量i存放第一个乘数，s和i相乘后将乘积重新放在s中，并作为新的第一个乘数，变量i的值增1，再与s相乘，依此类推，直到i的值大于5。

```
step1: s=1;  
step2 : i=1;  
step3 : s*i→s;  
step4 : i+1→i;  
step5 : 如果i小于等于5，执行step3以及之后的步骤；否则  
        执行下一步；  
step6: 输出s的值；  
step7: 算法结束。
```




4.2 算法的设计与分析

流程图:

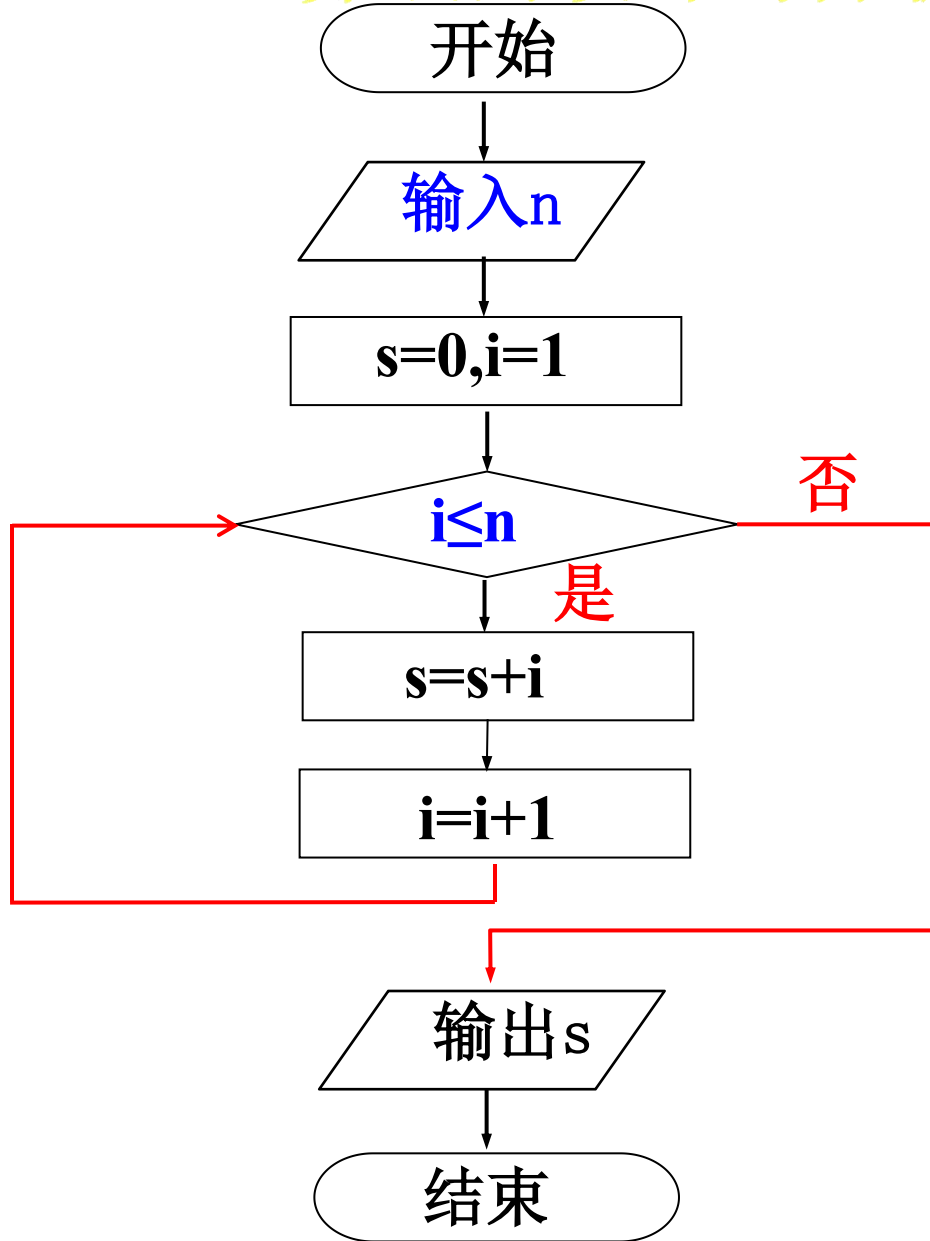




4.2 算法的设计与分析

4-4

计算 $1+2+3+\dots+n$,
 n 从键盘输入。
试画出流程图。





4.2 算法的设计与分析

4.2.4 常用的算法设计策略

1. 枚举法

枚举法，或称为穷举法、暴力破解法，其基本思路是：对于要解决的问题，**列举出所有可能的情况**，逐个判断哪个是符合条件要求的，从而得到问题的解。

如何从多把钥匙中找出那一把正确的钥匙？



一把一把地依次试。



4.2 算法的设计与分析

4.2.4 常用的算法设计策略

4-6

求1~1000中所有能被17整除的数。

问题分析：从1开始，逐一对每个数进行检验，看能否被17整除，直到大于1000为止---枚举法。

- 1) 初始化： $x=1$;
- 2) x 从1循环到1000;
- 3) 对于每一个 x ，依次进行检验：如果能被17整除，就打印输出，否则继续下一个数;
- 4) 重复第2至第3步，直到循环结束。



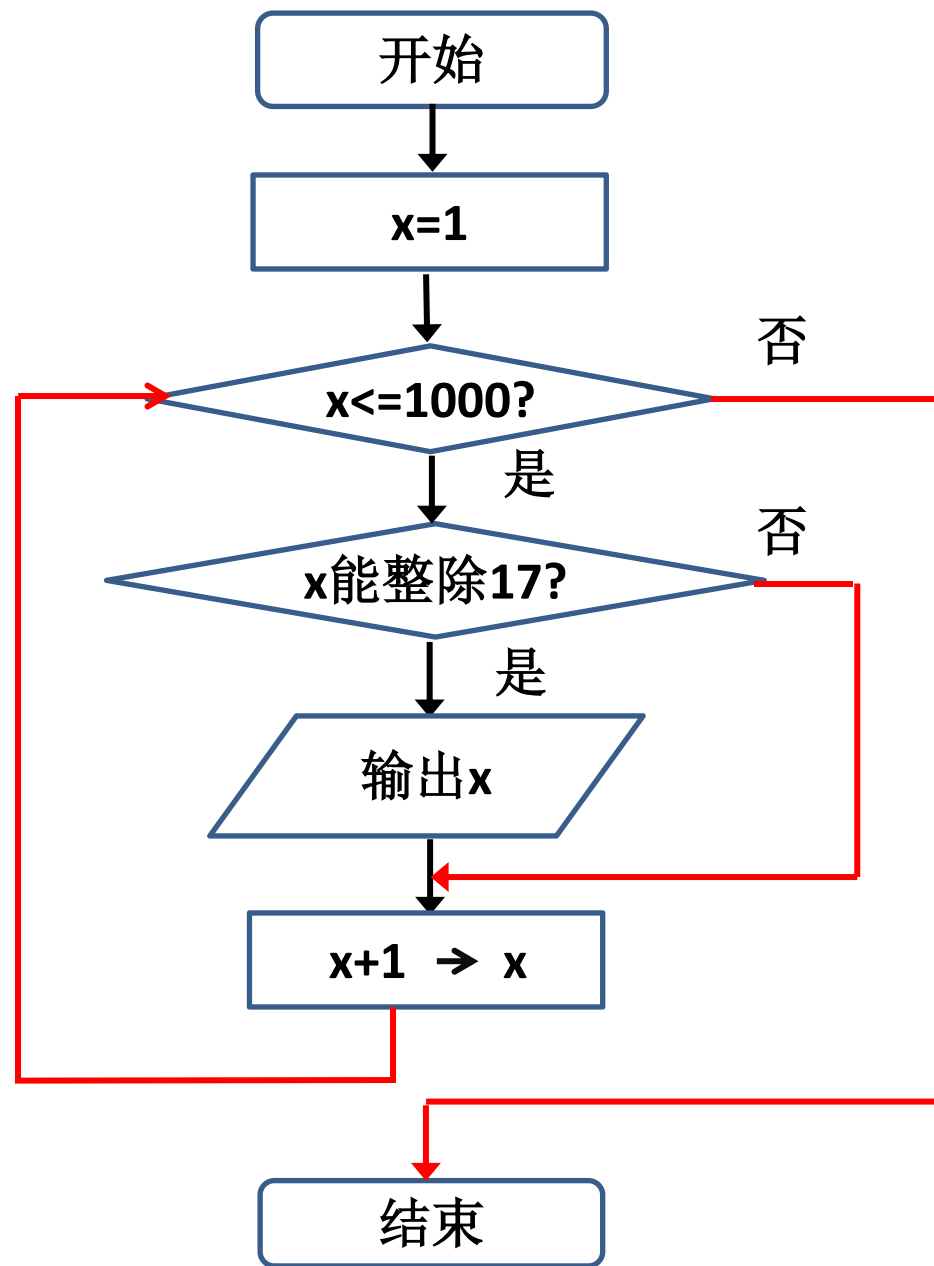
河北工程大学

Hebei University of Engineering

请同学们绘制流程图：

- 1) 初始化：x=1;
- 2) x从1循环到1000;
- 3) 对于每一个x，依次进行检验：如果能被17整除，就打印输出，否则继续下一个数;
- 4) 重复第2至第3步，直到循环结束。

枚举通常使用循环语句实现！





4.2 算法的设计与分析

4.2.4 常用的算法设计策略

4-7 穷举法：百钱买百鸡

公鸡每只5元，母鸡每只3元，小鸡一元三只，一百元买一百只鸡，问有几种买法？

【问题分析】设公鸡为 x 只，母鸡为 y 只，小鸡为 z 只，问题转化为一个三元一次方程组。

$$\begin{cases} x+y+z=100 \\ 5x+3y+z/3=100 \end{cases}$$

该不定解方程，将各种可能的取值依次代入，同时满足两个方程的值即问题的解。有哪些可能的取值呢？



公鸡	母鸡	鸡雏
1	1	98
1	2	97
...	...	...
1	33	66
2	1	97
2	2	96
...	...	...
2	33	65
20	1	79
20	2	78
...	...	...
20	33	47



4.2.4 常用的算法设计策略

4-7

百钱买百鸡

算法设计:

第1步: 初始化 $x=1$;

第2步: x 从1循环到20;

第3步: 对于每一个 x ,让 y 从1循环到33;

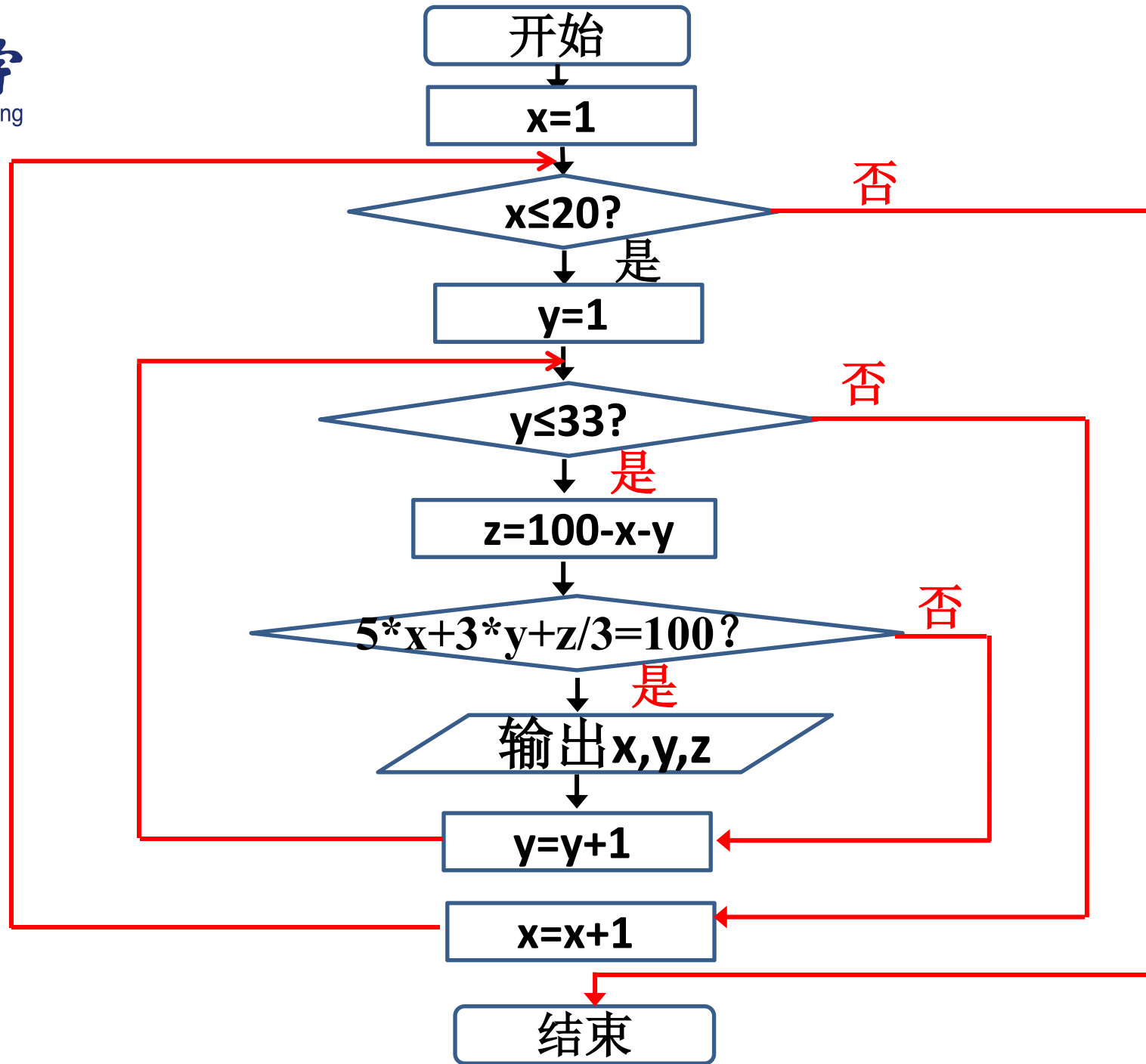
第4步: 在循环中, 对于上述每一个 x 和 y 值, 计算 $z=100-x-y$;

第5步: 如果 $5x+3y+z/3=100$ 成立, 就输出 x , y , z ;

第6步: 重复第2~5步, 直到循环结束。



百钱买百鸡





4.2.4 常用的算法设计策略

4-7

百钱买百鸡

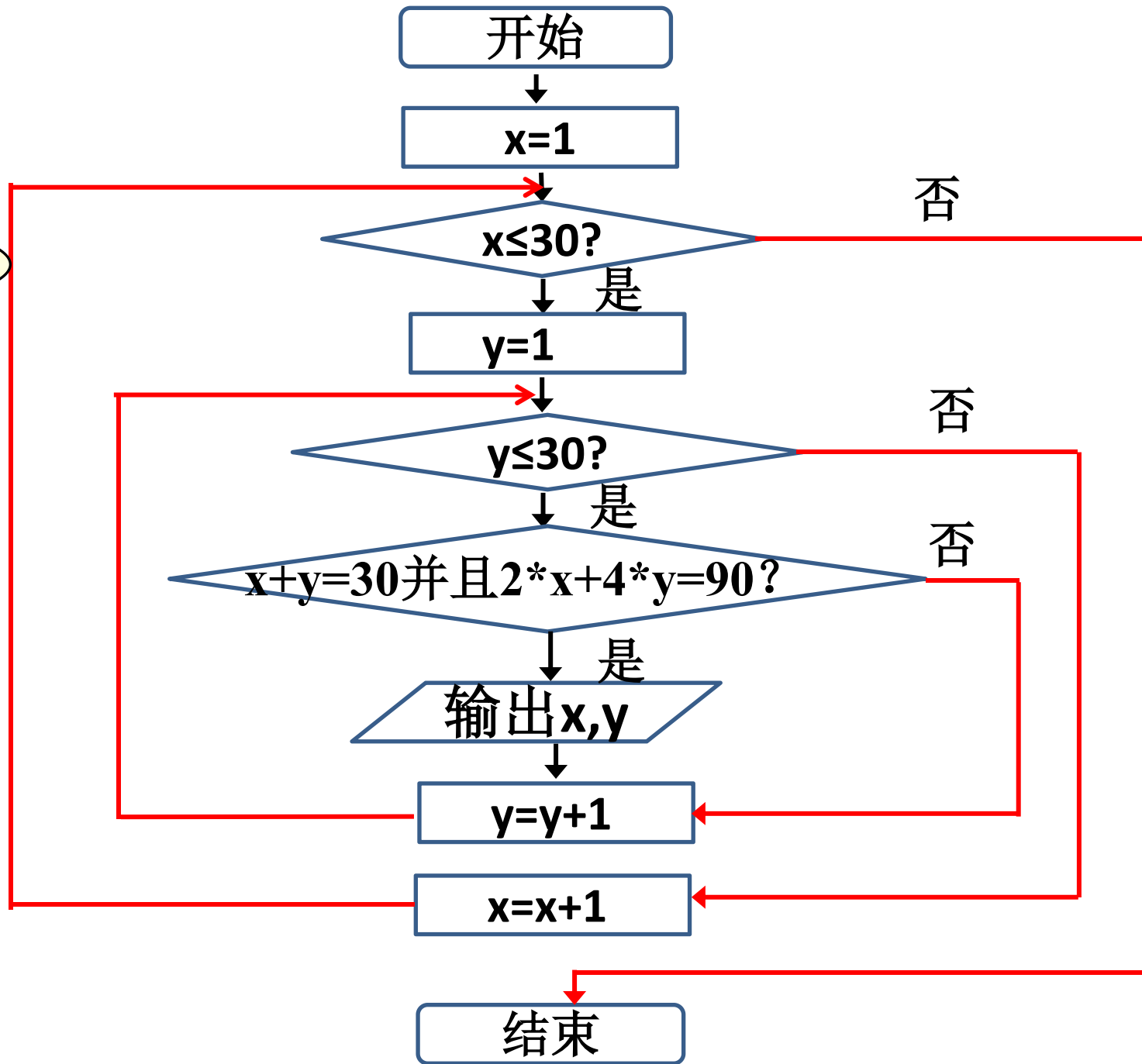
用Python语言实现，代码如下：

```
for x in range(1,21):  
    for y in range(1,34):  
        z=100-x-y  
        if(5*x+3*y+z/3==100):  
            print("公鸡: ",x,"母鸡: ",y,"小鸡: ",z)
```

公鸡:	4	母鸡:	18	小鸡:	78
公鸡:	8	母鸡:	11	小鸡:	81
公鸡:	12	母鸡:	4	小鸡:	84



鸡兔同笼





4-8

鸡兔同笼

用Python语言实现，代码如下：

```
for chicken in range(30):  
    for rabbit in range(30):  
        if (chicken+rabbit==30 and 2*chicken+4*rabbit==90):  
            print('鸡有%s只， 兔子有%s只'%(chicken,rabbit))
```

```
Python 3.5.2 Shell  
File Edit Shell Debug Options Window Help  
Python 3.5.2 (v3.5.2:4def2a2901a5, Jun  
25 2016, 22:18:55) [MSC v.1900 64 bit (AMD64)] on win32  
Type "copyright", "credits" or "license()  
()" for more information.  
>>>  
===== RESTART: C:/Users/Administrator/Desktop/1.py =====  
鸡有15只， 兔子有15只  
>>> |  
Ln: 6 Col: 4
```



4.2 算法的设计与分析

4.2.4 常用的算法设计策略

2. 递推法（迭代法、辗转法）

递推是按照一定的**规律**来计算序列中的每个项，通常是通过计算前面的一些项来得出序列中的指定项的值。

4-9

猴子吃桃问题

小猴摘了若干桃子，第1天吃掉一半多一个，第2天吃掉剩下的一半多一个，以后小猴每天吃掉前一天剩下桃子数的一半多一个，到第5天时只剩1个了，问小猴共摘多少个桃子？



问题分析:

用变量s存放桃子的数量

第5天: $s=1$

第4天: $s=2*(s+1)$

第3天: $s=2*(s+1)$

第2天: $s=2*(s+1)$

第1天: $s=2*(s+1)$

重复计算了4次
($n=5$ 天, 计算 $n-1$ 次)



第1天剩余	第2天剩余	第3天剩余	第4天剩余	第5天剩余
46	22	10	4	1

思路: 循环计算第4天到第1天桃子的数量。



4-9

猴子吃桃问题

自然语言描述算法如下：

第1步：初始化， $s=1, i=1$ ；

第2步：如果 $i \leq 4$ ，执行下一步；否则执行第5步；

第3步：计算前一天桃子的数量， $2*(s+1)$ 再存入 s 中；

第4步： i 的值自增1；返回至第2步重复执行；

第5步：输出 s ；

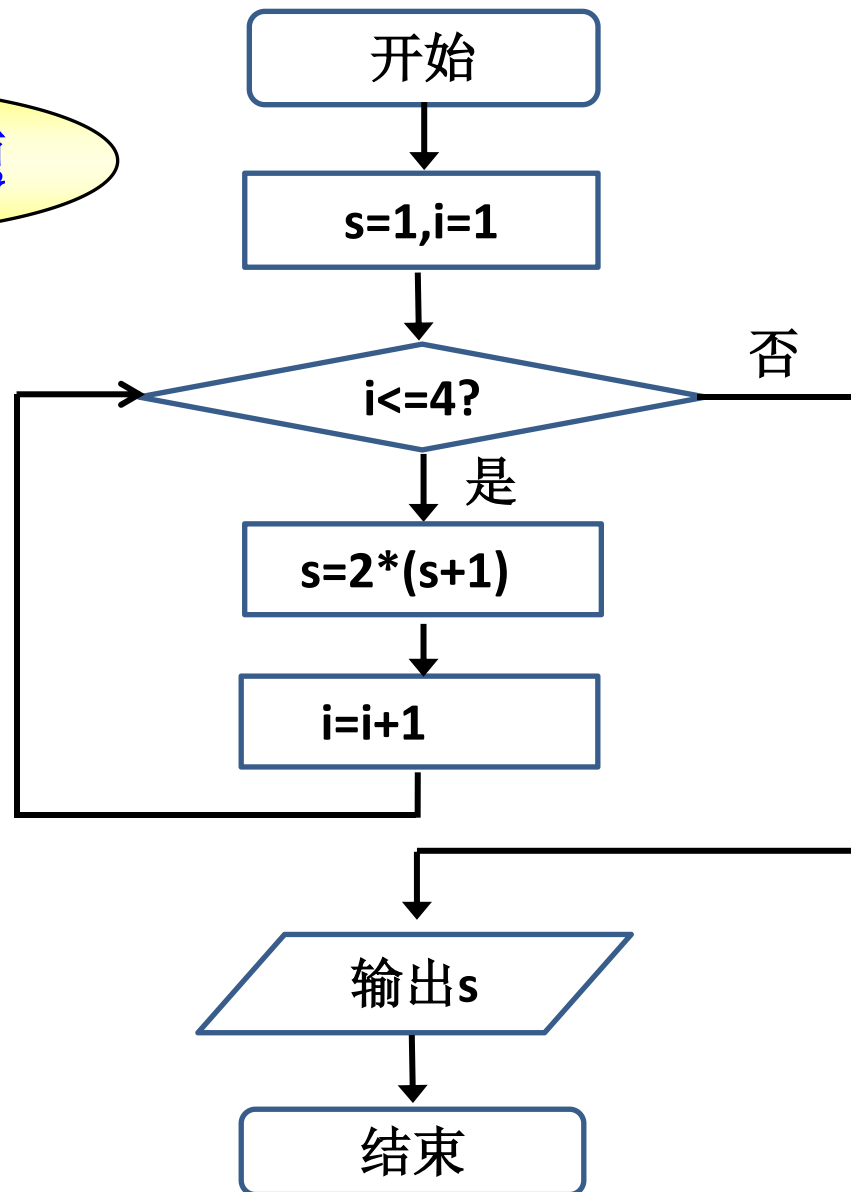
第6步：算法结束。



4-9

猴子吃桃问题

使用流程图描述算法？





用Python语言实现，代码如下：

```
s=1
for i in range(4):
    s=2*(s+1);
print('猴子一共摘了%d个桃子'%s);
```

```
Python 3.5.2 Shell
File Edit Shell Debug Options Window Help
Python 3.5.2 (v3.5.2:4def2a2901a5, Jun 25 2016, 22:18:55) [MSC v.1900 64 bit (AMD64)]
on win32
Type "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/Administ
rator/Desktop/1.py =====
猴子一共摘了46个桃子
>>>
```



小结

1. 什么是算法

为解决一个确定类问题而采取的方法和步骤。

2. 算法应具备的特征

确切性、可行性（有效性）、输入项、输出项、有穷性

3. 问题求解的步骤

(1) 问题抽象及数学建模

(2) 输入/输出

(3) 算法设计与分析

4. 算法的描述

自然语言、流程图、盒图（N-S图）、伪代码



4.2 算法的设计与分析

4.2.4 常用的算法设计策略

3. 递归法

对于一个较为复杂的问题，把原问题**分解**成若干个相对**简单且类同**的子问题；而简单到一定程度的子问题可以直接求解；这样，原问题就可递推得到解。

通过**调用自身**，只需少量的程序就可描述出多次重复计算。



4.2 算法的设计与分析

4-10 递归算法计算n!

$$n! = \begin{cases} 1 & n=1 \\ n*(n-1)! & n \geq 2 \end{cases}$$

【分析】

① $5! = 5 * 4!$

$4! = 4 * 3!$

$3! = 3 * 2!$

$2! = 2 * 1!$

$1! = 1$

② 递归公式可表示成：

$$\text{fac}(n) = n * \text{fac}(n-1)$$

递归算法关键：

- ◆ 递归的出口
- ◆ 递归公式



4.2 算法的设计与分析

4-10 递归算法计算n!

$$n! = \begin{cases} 1 & n=1 \\ n*(n-1)! & n \geq 2 \end{cases}$$

递归算法描述如下：

①输入n。

②**递推阶段**：计算n!，需先计算(n-1)!，而计算(n-1)!，又需先计算(n-2)!依次类推，直至计算出1! 等于1。

③**回归阶段**：得到1! 等于1后，根据2! =2*1! ，返回得到2!依次步步回归，在得到(n-1)! 的结果后，得到n! 。

④输出n!。



4.2 算法的设计与分析

4.2.4 常用的算法设计策略

4-11 使用递归法解决斐波那契数列问题。

斐波那契（Fibonacci，约1170-1250）是意大利著名数学家。在他的著作《算盘书》中有许多有趣的问题，最著名的问题是的“兔子繁殖问题”。

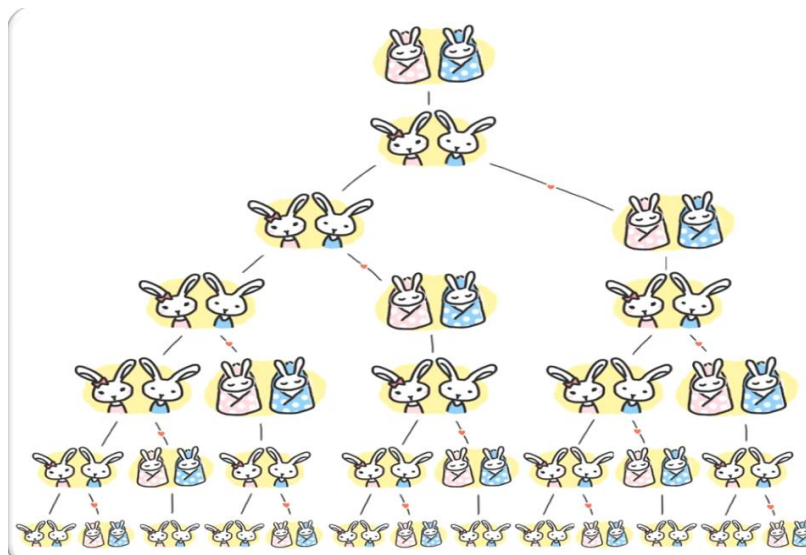
如果一对大兔每月能生1对小兔，而每对小兔在它出生后的第3个月就长成为大兔。那么，由1对初生的小兔开始，12个月后能繁殖成多少对兔子？



4.2 算法的设计与分析

4.2.4 常用的算法设计策略

如果一对大兔每月能生1对小兔，而每对小兔在它出生后的第3个月就长成为大兔。



数学模型：无穷数列1, 1, 2, 3, 5, 8, 13, 21, 34, 55, ..., 被称为Fibonacci数列。



4.2.4 常用的算法设计策略

Fibonacci数列1, 1, 2, 3, 5, 8, 13, 21, 34....。

它可以递归地定义为:

$$F(n)=\begin{cases} 1 & n=1 \\ 1 & n=2 \\ F(n-1)+F(n-2) & n>2 \end{cases}$$

如计算第5项的值:

$$F(5)=F(4)+F(3)$$

$$F(4)=F(3)+F(2)$$

$$F(3)=F(2)+F(1)$$

$$F(1)=1; F(2)=1;$$

◆ 递归的出口
 $n=1,2$

◆ 递归公式
 $F(n)=F(n-1)+F(n-2)$



4.2.4 常用的算法设计策略

4-11 使用递归法解决斐波那契数列问题。

$$F(n) = \begin{cases} 1 & n = 1 \\ 1 & n = 2 \\ F(n-1) + F(n-2) & n > 2 \end{cases}$$

算法如下：

- ①递推阶段：求解 $F(12)$ ，必须先计算 $F(11)$ 和 $F(10)$ ，而得到 $F(11)$ 和 $F(10)$ 的值，又必须先计算 $F(9)$ 和 $F(8)$ 。依此类推，直至计算 $F(2)$ 和 $F(1)$ ，得到结果1和1。
- ②回归阶段：得到 $F(2)$ 和 $F(1)$ 后，返回得到 $F(3)$ 和 $F(4)$ ，……，在得到了 $F(10)$ 和 $F(11)$ 后，返回得到 $F(12)$ 。
- ③输出 $F(12)$ 。



4.2 算法的设计与分析

4.2.4 常用的算法设计策略

4. 回溯法（试探法）

“枚举-试探-失败返回-再枚举试探”的求解方法就称为回溯法。



迷宫游戏：

所有可能解的空间进行搜索，算法复杂度较高。类似枚举的搜索尝试过程，在搜索过程中寻找问题的解，当发现不满足求解条件时，就“回溯”返回，尝试别的路径。

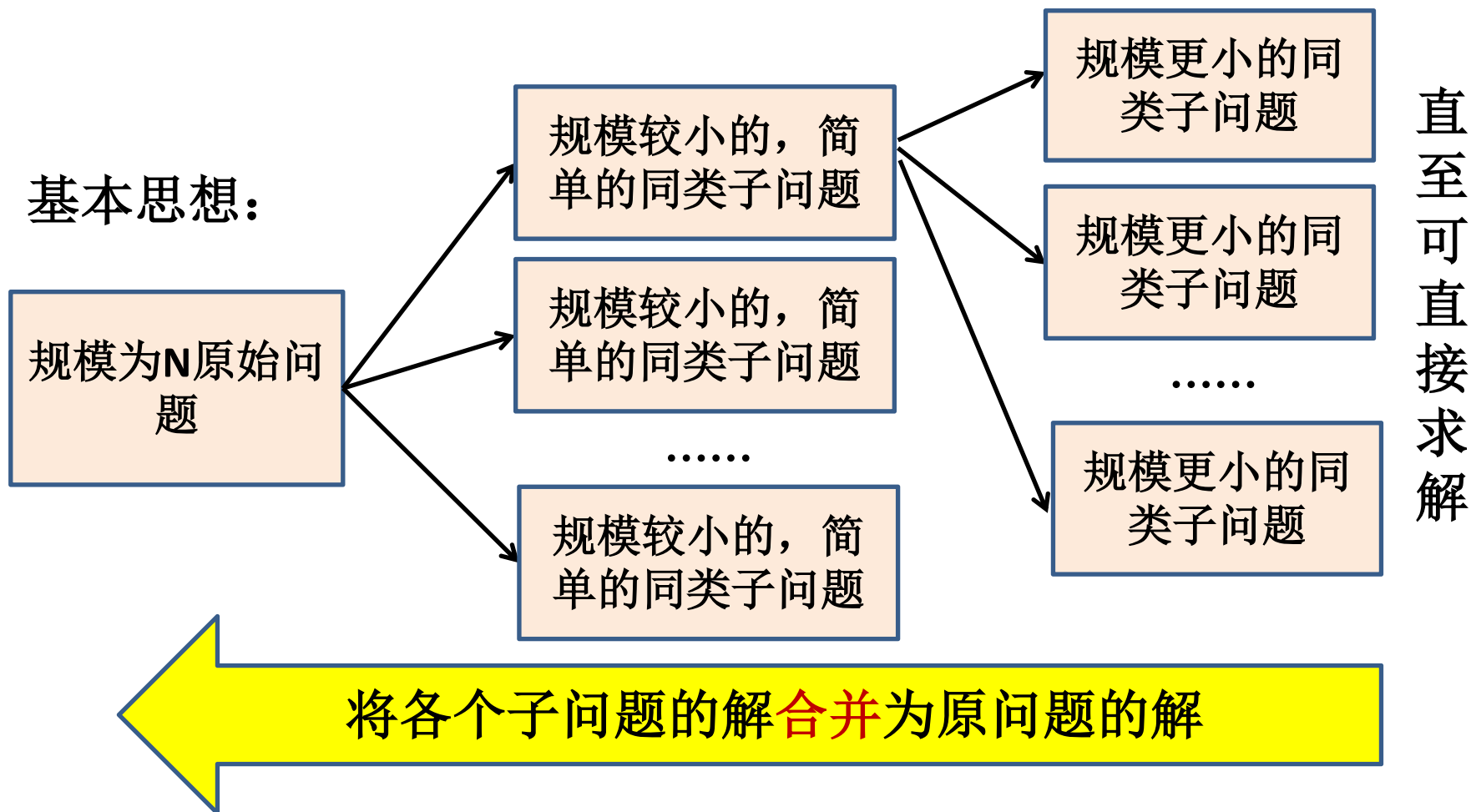
为了提高效率，应该充分利用给出的约束条件，尽量避免不必要的试探。



4.2 算法的设计与分析

5. 分治法

分而治之，是处理复杂问题的一个最基本、最常用的策略之一。





4.2 算法的设计与分析

4.2.4 常用的算法设计策略

4-14 公主的婚姻

从前一个年轻的国王向邻国一个聪明美丽的公主求婚，公主出了一道题：求48770428433377171的一个真因子（除它本身和1外的其他约数）？若国王能在一天之内求出答案，便接受他的求婚。



他能成功
娶到公主吗？



4.2.4 常用的算法设计策略

【算法分析】该数为17位，则最小的一个真因子不会超过9位，宰相给国王出了一个主意：按自然数的顺序给全国的老百姓每人编一个号发下去，等公主给出数目后，立即将它们通报全国，让每个老百姓用自己的编号去除这个数，除尽了立即上报，赏金万两。

□ 宰相提出的是分治法。





4.2 算法的设计与分析

常见算法设计策略(掌握前三种):

- 枚举法
- 递推法
- 递归法
- 回溯法
- 分治法





4.2 算法的设计与分析

4.2.5 算法分析

1.对算法进行全面分析，可分两个阶段进行：

(1) 事前分析

指通过对算法本身的执行性能的理论分析，得出算法特性。

算法复杂性 = 算法所需要的计算机资源：时间复杂度和空间复杂度

(2) 事后测试

正确性、可读性、健壮性、复杂性（高低和算法所需要的**计算机资源**成正比）



4.2 算法的设计与分析

4.2.5 算法分析

2. 评价算法的性能指标

- (1) 正确性
- (2) 可读性
- (3) 健壮性（容错性）：对不同的输入都要有相应的反应，比如合法的输入就要有相应的输出，不合法的输入要有相应的提示信息输出。
- (4) 时间复杂度：执行算法所需要的计算工作量。
- (5) 空间复杂度：指算法需要消耗的内存空间。



4.3 算法的实现——程序设计语言

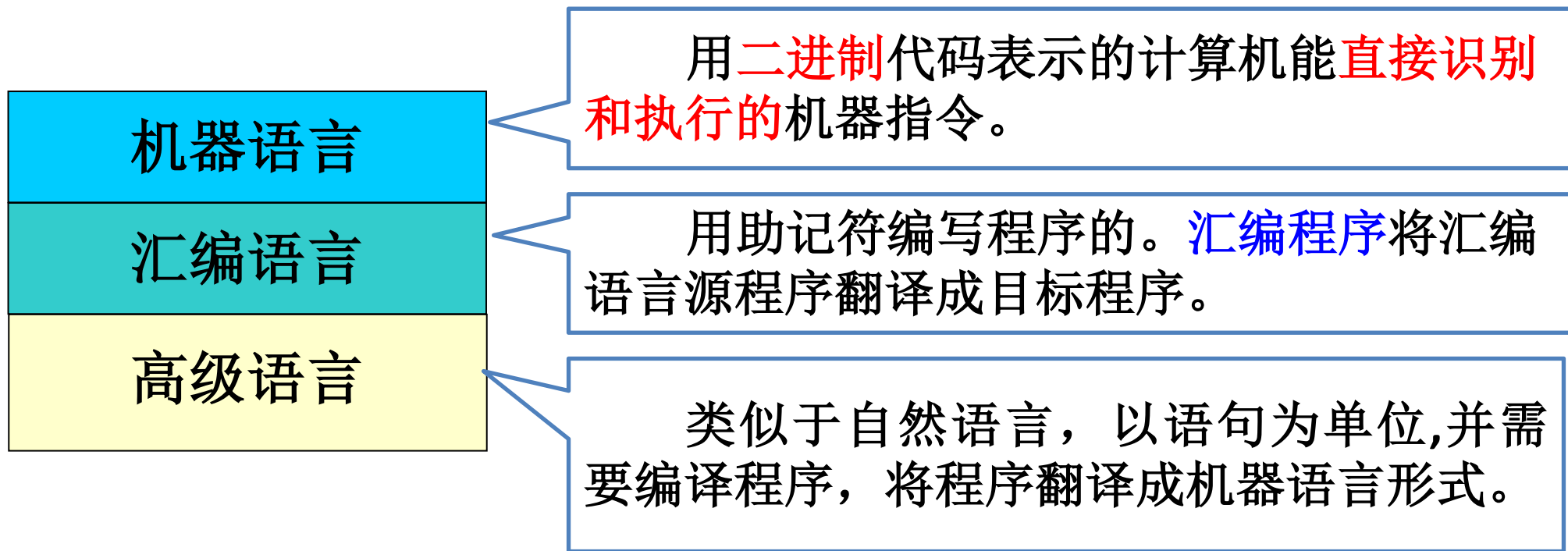
4.3.1 程序设计语言的分类

4.3.2 语言处理程序

4.3.3 常用的高级语言*



4.3.1 程序设计语言的分类





4.3 算法的实现——程序设计语言

程序设计语言----机器语言

机器语言是用**二进制**代码方式表示的计算机能**直接识别和执行**的一种机器指令的集合。

所有的程序都需转换成**机器语言**程序，计算机才能执行。

100001	10
000010	11
100010	10
000010	10
100101	11
000001	10
111101	00

计算7+10并存储



4.3 算法的实现——程序设计语言

程序设计语言——汇编语言

- ◆ 用助记符编写程序的语言；
- ◆ 汇编语言提供了助记符编写程序的规范/标准，同时开发了一个翻译程序，实现了将符号程序自动转换成机器语言程序的功能。
- ◆ 用符号编写程序->翻译->机器语言程序

操作码

地址码

100001

1000000111



MOV A,7

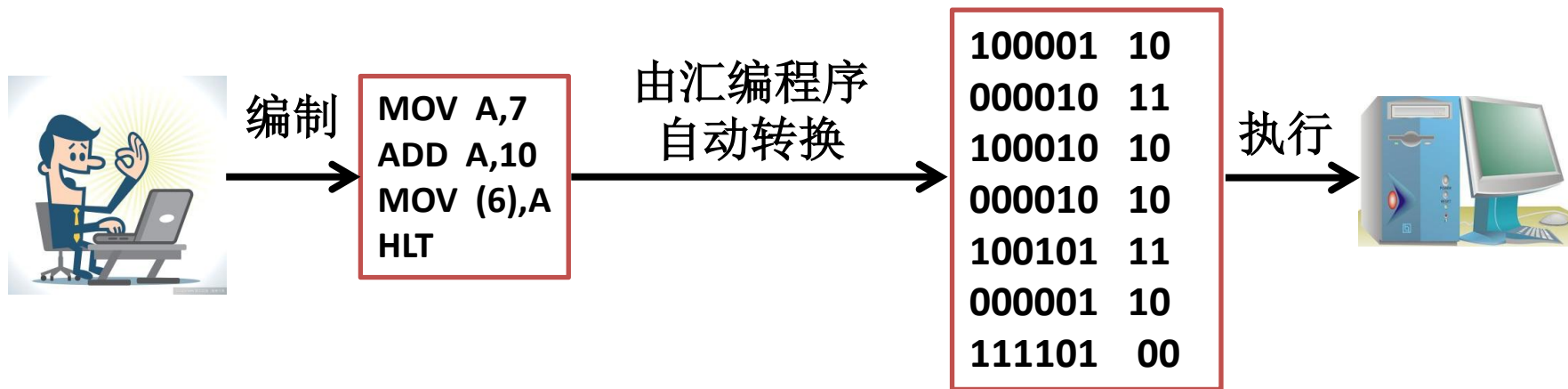
计算7+10并存储

```
MOV A,7  
ADD A,10  
MOV (6),A  
HLT
```



4.3 算法的实现——程序设计语言

- 汇编语言源程序：用汇编语言编写的程序。
- 汇编程序：将汇编语言源程序翻译成机器语言程序的程序。



汇编程序即转换规则（助记符----机器指令）



4.3 算法的实现——程序设计语言

程序设计语言——高级语言

类似于自然语言方式，以语句为单位书写程序的规范/标准。并开发了一个翻译程序（编译程序），实现了将语句程序自动翻译成机器语言程序的功能。

- ◆ 高级语言源程序：用高级语言编写的程序。
- ◆ 编译程序：将高级语言源程序翻译成机器语言程序的程序。

两种方式：解释方式、编译方式。

程序设计语言——非过程化语言（SQL）

计算7+10并存储

```
sum=7+10;  
return;
```



4.3 算法的实现——程序设计语言

4.3.2 语言处理程序

- ◆ 汇编程序

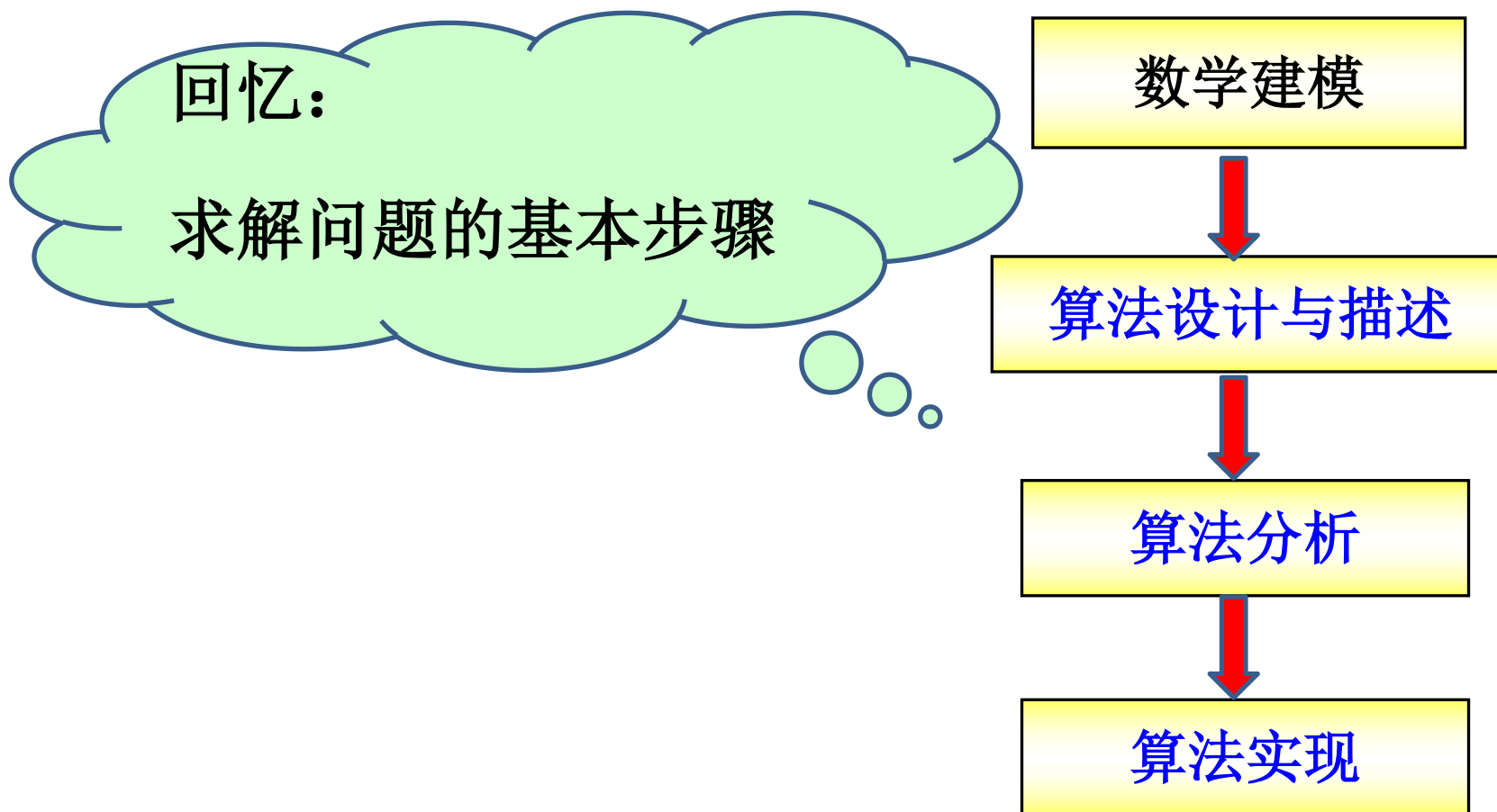
汇编程序将汇编语言源程序翻译成目标程序。

- ◆ 编译程序

将高级语言源程序翻译成目标程序。



4.3 算法的实现——程序设计语言





河北工程大学
Hebei University of Engineering

算法设计练习



1.某单位发放临时工资，工人每月工作不超过20天时，一律发放¥2000，超过20天分段处理，25天以内超过天数，每天¥100，25天以上，每天¥150。设计一个算法根据输入的天数计算，应发的工资，并画出程序框图。

$$\left\{ \begin{array}{l} 1\sim 20\text{天（含20天）：} 2000 \\ 20\sim 25\text{天（含25天）：} 2000+(d-20)*100=100d \\ 25\text{天以上：} 2000+5*100+(d-25)*150=150d-1250 \end{array} \right.$$

2.输入n（正整数），将1至n以内所有能被7和11整除的数输出。（使用程序流程图描述算法）



算法设计练习

3. 计算 $1+2+4+\dots+2^n$ (通项是 2^i)。

4. 设计算法, 找出由 n 数组成的数列 x 中最大的数 Max 。

5. 递归算法计算 $n!$

6. 一张单据上有一个5位数的编号, 万位数是1, 千位数是4, 百位数是7, 个位数、十位数已经模糊不清 ($147XY$)。该5位数是57或67的倍数, 输出所有满足这些条件的5位数的个数, 设计求解该问题的算法。

7. 雨水淋湿了算术书的一道题, 8个数字只能看清3个, 第一个数字虽然看不清, 但可看出不是1。设计一个算法求其余数字是什么?

$$[\square \times (\square 3 + \square)]^2 = 8\square\square 9$$

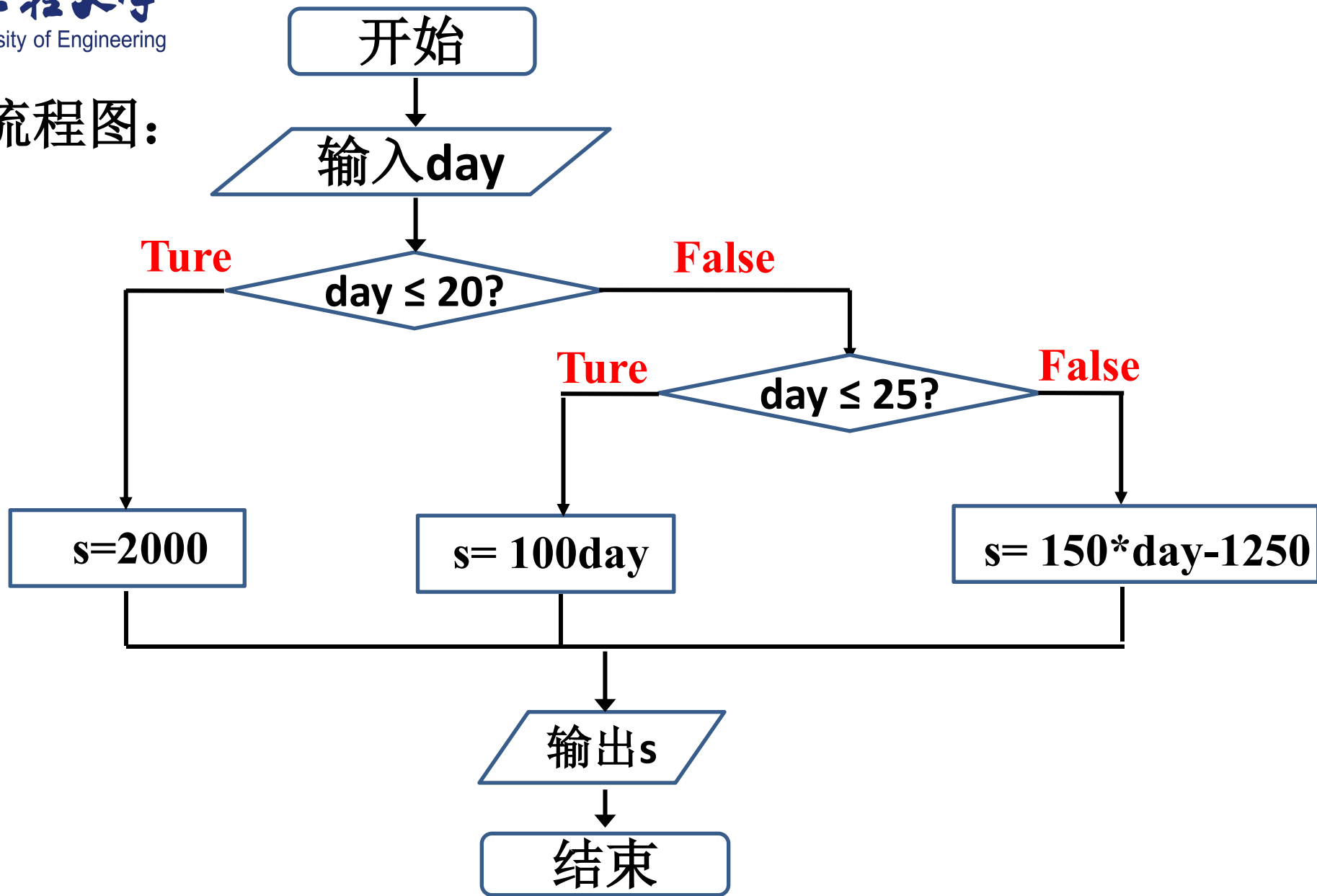


算法设计练习

8. 有5个人，第5个人说他比第4个人大2岁，第4个人说他对第3个人大2岁，第3个人说他比第2个人大2岁，第2个人说他比第1个人大2岁，第1个人说他10岁。求第5个人多少岁。
9. 有个莲花池里起初有一只莲花，每过一天莲花的数量就会翻一倍。假设莲花永远不凋谢，30天的时候莲花池全部长满了莲花，请问第23天的莲花占莲花池的几分之几？设计本问题的算法。

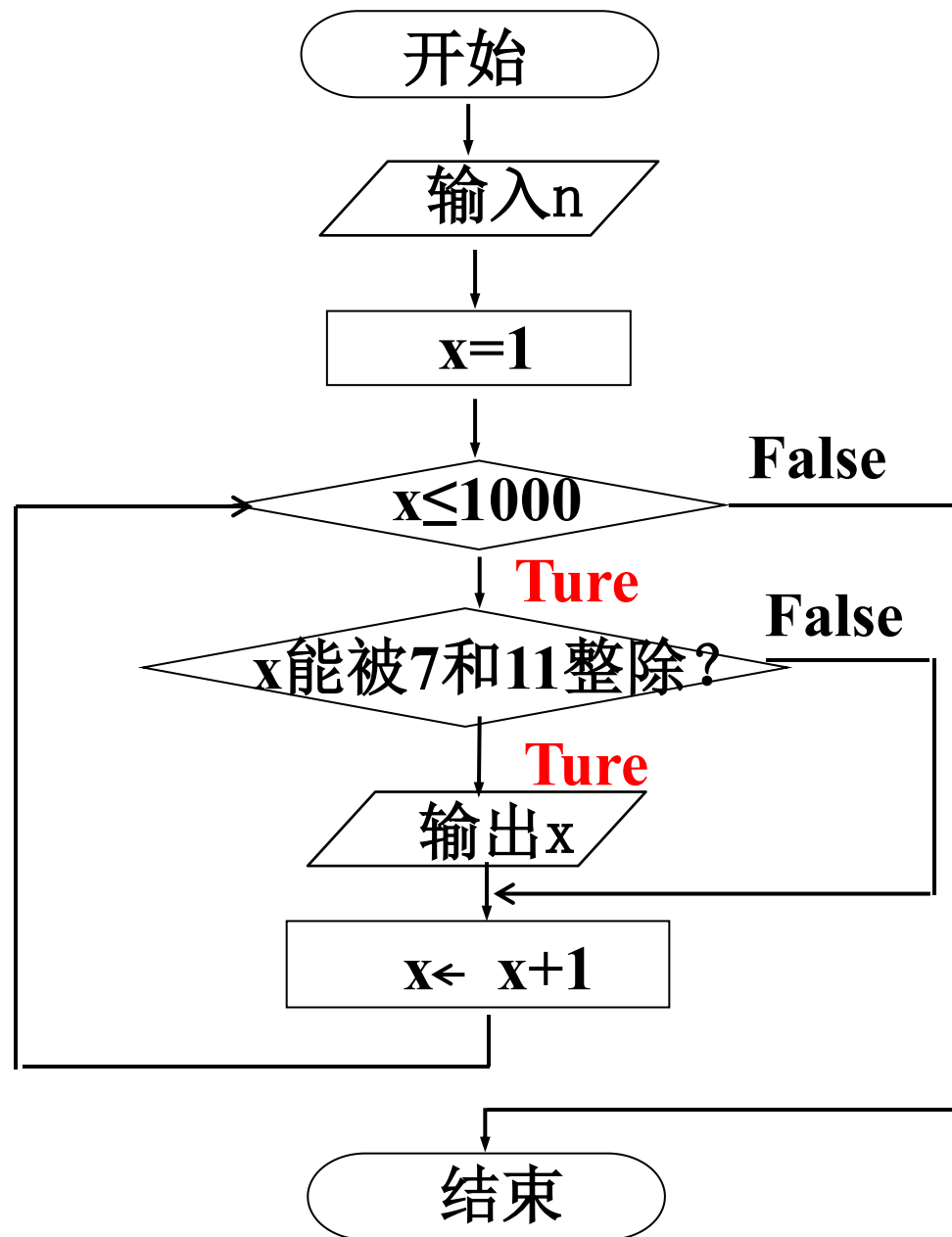


1.流程图:



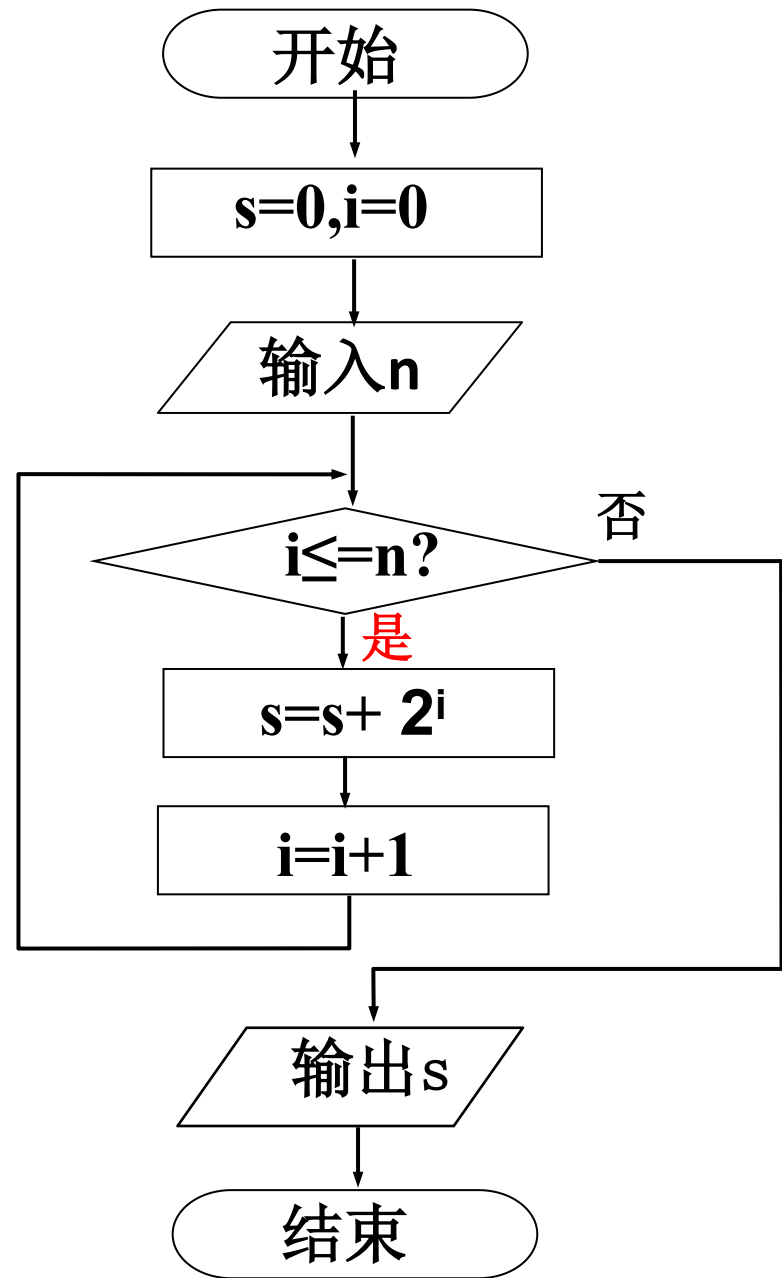


2.流程图:





3. 计算 $1+2+4+\dots+2^n$ (通项是 2^i) 。





4.设计算法，找出由 n 数组成的数列 x 中最大的数 $\max$ 。

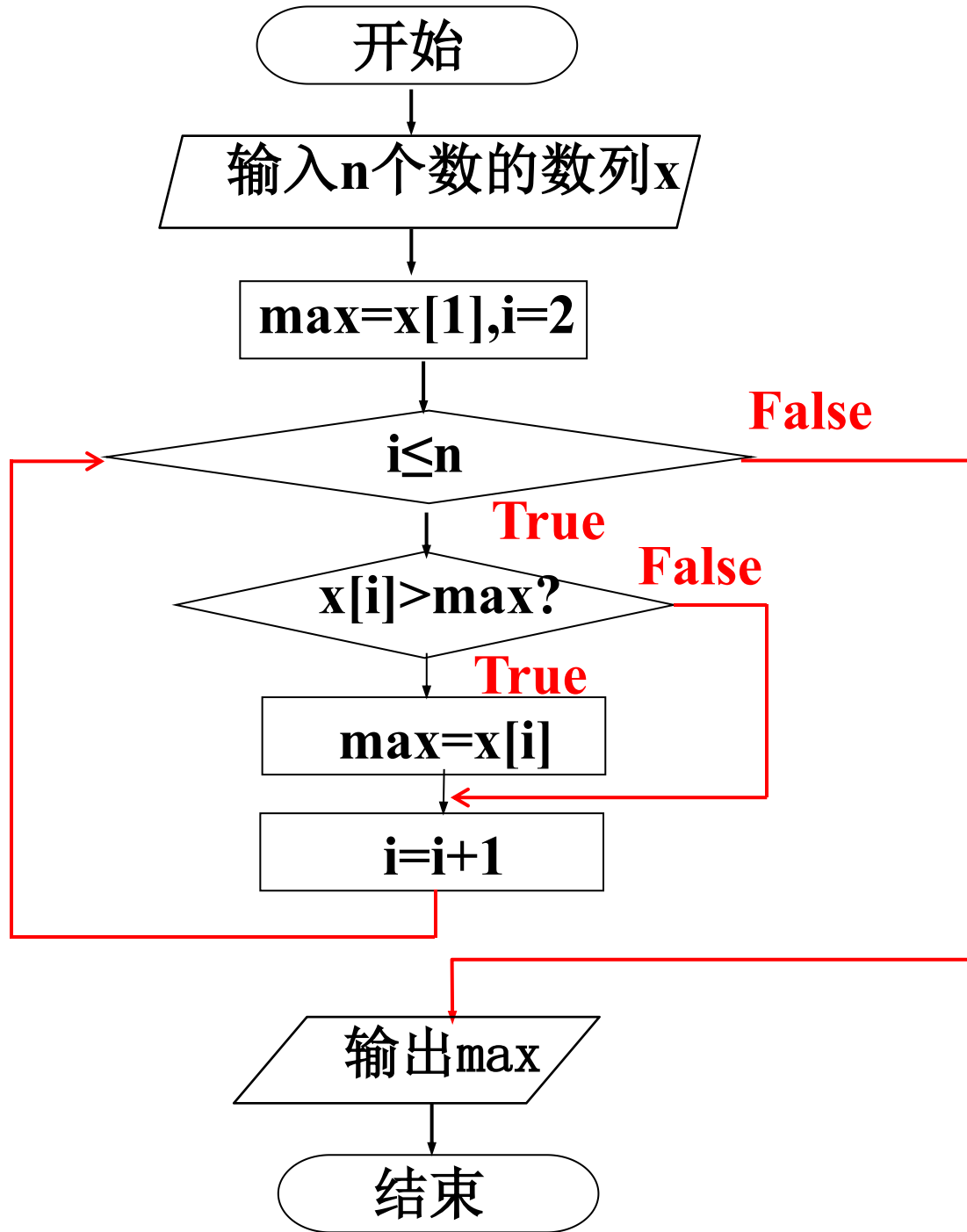
分析：

捡豆子----如果将数列中的每一个数字大小看成是一颗豆子的大小。

首先将第一颗豆子放入口袋中；从第二颗豆子开始比较，如果正在比较的豆子比口袋中的还大，则将它捡起放入口袋中，同时丢掉原先口袋中的豆子，如此循环直到最后一颗豆子；最后口袋中的豆子就是所有的豆子中最大的一颗。



4.设计算法，找出由n个数组成的
数列x中最大的数max。





5.递归算法计算n!
$$n! = \begin{cases} 1 & n=1 \\ n*(n-1)! & n \geq 2 \end{cases}$$

递归算法描述如下：

- 1) 输入n的值。
- 2) 递推阶段：因为 $n!=n*(n-1)!$ ，所以计算F(n)，必须先计算F(n-1)，而计算F(n-1)，又必须先计算F(n-2)。依次类推，直至计算F(1)，立即得到结果1。
- 3) 回归阶段：得到F(1)后，返回得到F(2)的结果，.....，在得到了F(n-1)的结果后，返回得到F(n)的结果。
- 4) 输出F(n) 的值。



6. 一张单据上有一个5位数的编号，万位数是1，千位数是4，百位数是7，个位数、十位数已经模糊不清（147XY）。该5位数是57或67的倍数，输出所有满足这些条件的5位数的个数，设计求解该问题的算法。

【算法1分析】 设x、y变量分别表示十位数和个位数。其取值范围为0~9。

- 1) $n=0$
- 2) x从0循环到9;
- 3) 对于每一个x，依次地让y从0循环到9;
- 4) 在循环中，对于上述每一个x、y变量值，如果组成的5位数 $(14700+10*x+y)$ 是57或67的倍数，则 $n=n+1$, 否则执行下一步;
- 5) 重复第2至第4步，直到循环结束。
- 6) 输出n。

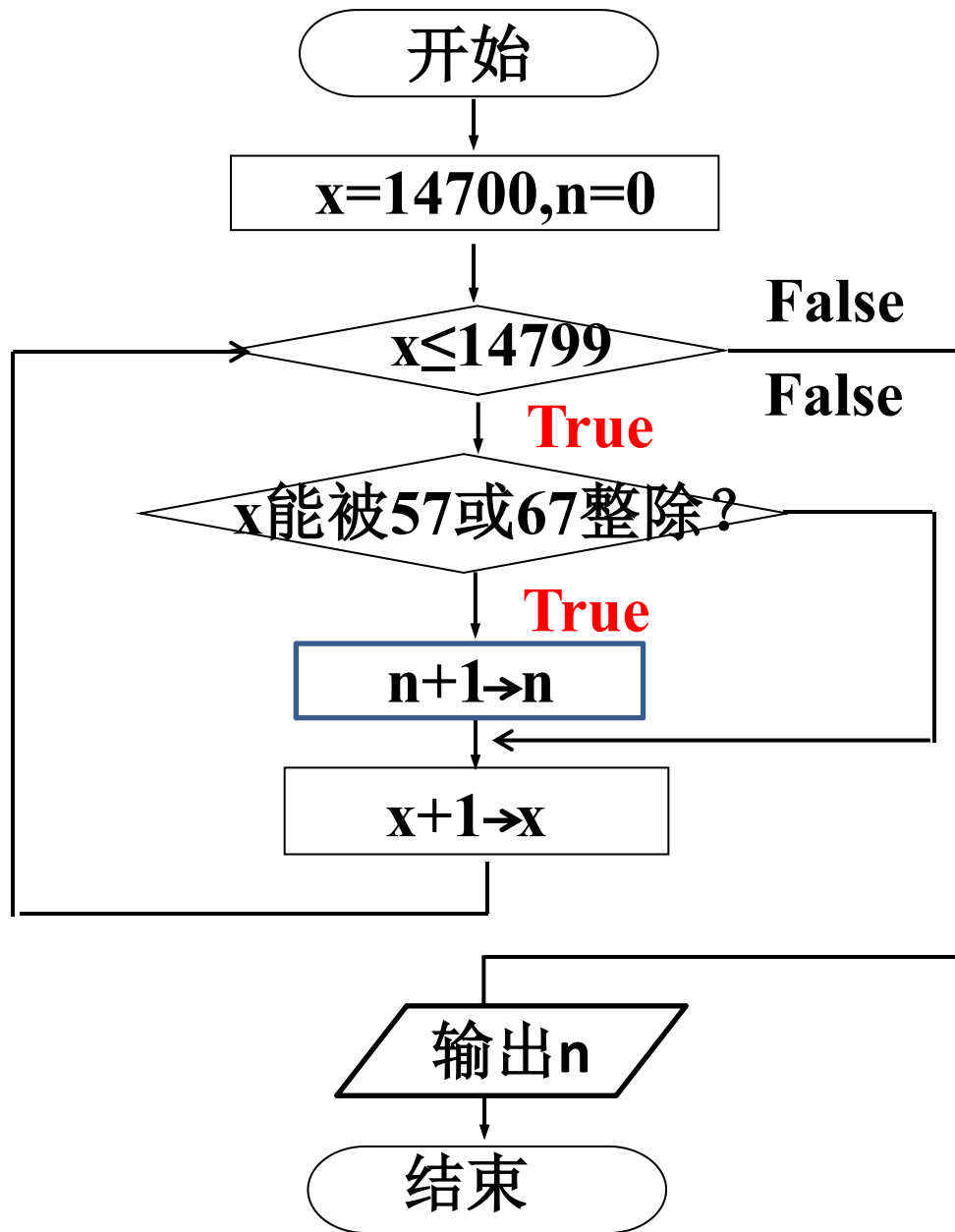


【算法2分析】

- 1) $n=0$
- 2) x 从14700循环到14799;
- 3) 对于上述每一个 x , 如果是57或67的倍数, 则 $n=n+1$, 否则执行下一步;
- 4) 重复第2、第3步, 直到循环结束。
- 5) 输出 n 。



流程图:





7. 雨水淋湿了算术书的一道题，8个数字只能看清3个，第一个数字虽然看不清，但可看出不是1。设计一个算法求其余数字是什么？

$$[\square \times (\square 3 + \square)]^2 = 8\square\square 9$$

【分析】设a、b、c、d、e五个变量分别表示自左到右5个未知的数字。其中a的取值范围为2~9，b的取值范围为1~9，c的取值范围为1~9，d和e的取值范围为0~9。判断条件即题目给出的算式。



自然语言描述算法步骤如下：

- 1) a从2循环到9;
- 2) 对于每一个a, 依次地让b从1循环到9;
- 3) 对于每一个b, 依次地让c从1循环到9;
- 4) 对于每一个c, 依次地让d从0循环到9;
- 5) 对于每一个d, 依次地让e从0循环到9;
- 6) 在循环中, 对于上述每一个a、b、c、d、e五个变量值, 如果满足 $(a*(b*10+3+c))^2=8009+d*100+e*10$, 就输出当前a、b、c、d、e五个变量的值;
- 7) 重复第1至第6步, 直到循环结束。



8. 有5个人，第5个人说他比第4个人大2岁，第4个人说他对第3个人大2岁，第3个人说他比第2个人大2岁，第2个人说他比第1个人大2岁，第1个人说他10岁。求第5个人多少岁。

利用**递归法**:
$$\text{age}(n) = \begin{cases} 10 & (n=1) \\ \text{age}(n-1)+2 & (n>1) \end{cases}$$

递归算法描述如下:

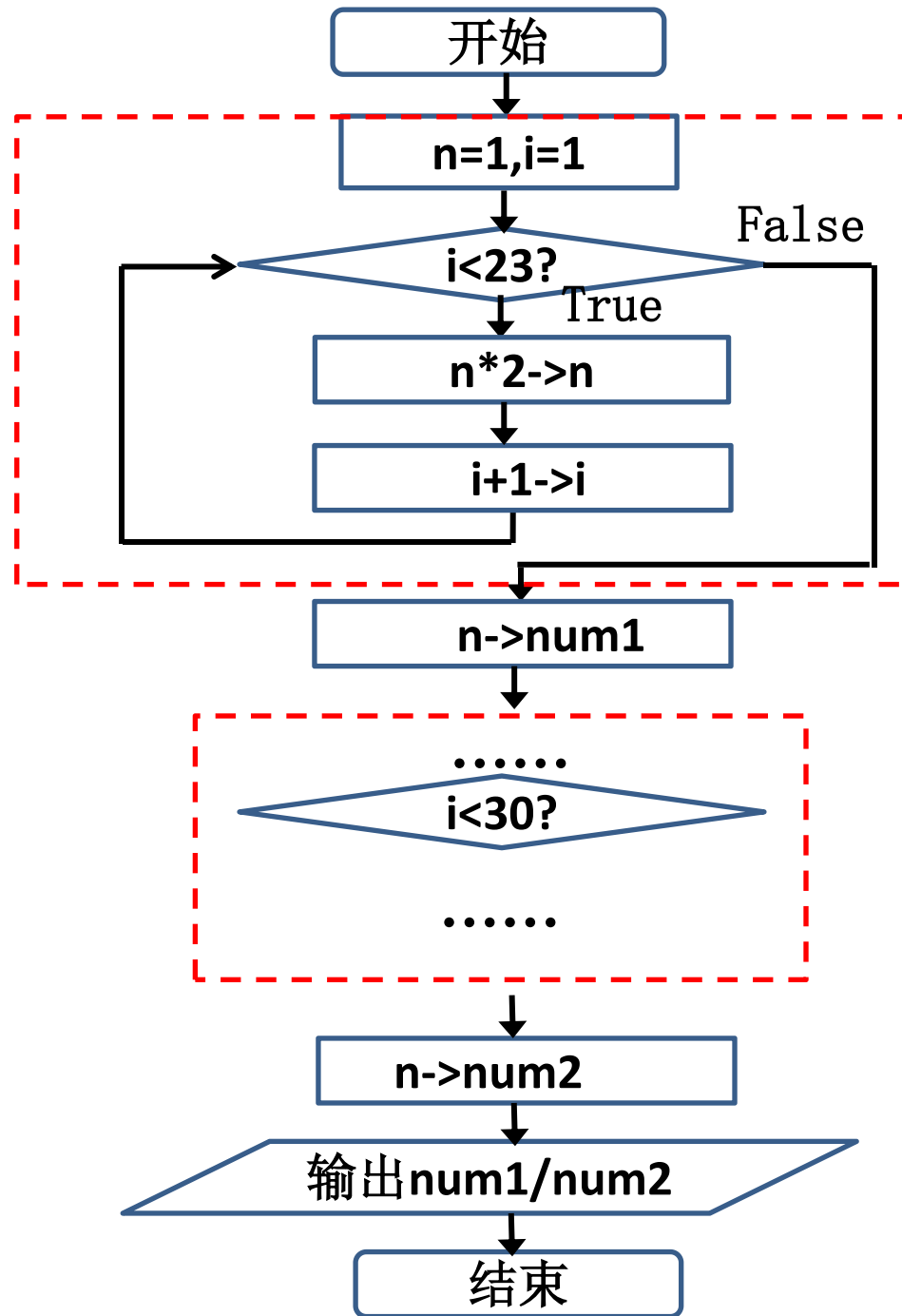
①递推阶段: 计算 $\text{age}(5)$ ，需先计算 $\text{age}(4)$ ，而计算 $\text{age}(4)$ ，又需先计算 $\text{age}(3)$ 依次类推，直至计算出 $\text{age}(1)=10$ 。

②回归阶段: 得到 $\text{age}(1)=10$ 后，根据 $\text{age}(2)=\text{age}(1)+2$ ，得到 $\text{age}(2)=12$ ，依次步步回归，在得到 $\text{age}(4)$ 后，根据 $\text{age}(5)=\text{age}(4)+2$ 得到 $\text{age}(5)$ 。



9.有个莲花池里起初有一只莲花，每过一天莲花数量就会翻一倍。假设莲花永远不凋谢，30天的时候莲花池全部长满了莲花，请问第23天的莲花占莲花池的几分之几？设计本问题的算法。

①递推法





②递归法： 定义递归公式： $F(n)=F(n-1)*2。$ ；
定义递归终止条件 $F(1)=1$

自然语言描述：

- ① 递推阶段：求解 $F(23)$ ，把它递推到求解 $F(22)$ ，而计算 $F(22)$ ，又必须先计算 $F(21)$ 。依次类推，直至计算 $F(1)$ 时，立即得到结果1。
- ② 回归阶段：得到 $F(1)$ 后，返回得到 $F(2)$ 的结果，.....，在得到了 $F(22)$ 的结果后，返回得到 $F(23)$ 的结果。
- ③ 求解 $F(30)$ 。
- ④ 计算 $F(23)/F(30)$ ，并输出。



10. 有一个农场在第一年的时候买了一头刚出生的牛，这头牛在第四年的时候就能生一头小牛，以后每年这头牛就会生一头小牛。这些小牛成长到第四年又会生小牛，以后每年同样会生一头牛，假设牛不死，如此反复。请问50年后，这个农场会有多少头牛？设计本问题的算法。

【分析】第四年后,牛的数量由两部分组成,去年的所有牛+所生下的小牛组成;由于不是所有牛都能生小牛,只有3年前的牛第4年才会生小牛,即 $f(n-3)$ 。设 n 代表牛的年龄, $f(n)$ 表示第 n 年的牛数, 可以递归地定义为:

$$F(n) = \begin{cases} 1 & (n=1) \\ 1 & (n=2) \\ 1 & (n=3) \\ F(n-1) + F(n-3) & (n \geq 4) \end{cases}$$



自然语言描述:

- 1) 递推阶段: 求解 $F(n)$, 把它递推到求解 $F(n-1)+F(n-3)$, 而计算 $F(n-1)$ 和 $F(n-3)$, 又必须先计算 $F(n-2)$ 、 $F(n-4)$ 、 $F(n-6)$ 。依次类推, 直至递推至计算 $F(3)$ 、 $F(2)$ 、 $F(1)$ 时, 得到值均为1。
- 2) 回归阶段: 得到 $F(3)$ 、 $F(2)$ 、 $F(1)$ 后, 返回得到 $F(4)$, 得到 $F(5)$,, 得到 $F(50)$ 的值。
- 3) 输出 $F(50)$ 。



河北工程大学
Hebei University of Engineering

本章结束